

A
Mini Project Report
On
**ADVANCED THERMAL IMAGING TECHNIQUES FOR METAL
RECOGNITION**

Submitted to CMR ENGINEERING COLLEGE

In Partial Fulfillment of the requirements for the award of degree of

**BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY**

Submitted
By

A. ARAVIND	238R1A1201
MD. WASEEM	238R1A1237
N. ANUSRI	238R1A1242
S. BHAVANA	238R1A1254

Under the Esteemed guidance of

Dr. E. SURESH BABU

Associate Professor, Department of IT



**Department of Information Technology
CMR ENGINEERING COLLEGE
(UGC AUTONOMOUS)**

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)

(Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

(2025-2026)

CMR ENGINEERING COLLEGE

(UGC AUTONOMOUS)

(Accredited by NAAC & NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)

(Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

Department of Information Technology



CERTIFICATE

This is to certify that the project entitled “**ADVANCED THERMAL IMAGING TECHNIQUES FOR METAL RECOGNITION**” is a Bonafide work carried out by

A. ARAVIND	238R1A1201
MD. WASEEM	238R1A1237
N. ANUSRI	238R1A1242
S. BHAVANA	238R1A1254

in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **INFORMATION TECHNOLOGY** from CMR Engineering College, affiliated to JNTU, Hyderabad, under our guidance and supervision.

The results presented in this project have been verified and are found to be satisfactory. The results embodied in this project have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide

DR. E. SURESH BABU

Associate Professor

Department of IT

CMREC, Hyderabad

Head of the Department

Dr. E. SURESH BABU

Associate Professor & HOD

Department of IT

CMREC, Hyderabad

DECLARATION

This is to certify that the work reported in the present project entitled “**ADVANCED THERMAL IMAGING TECHNIQUES FOR METAL RECOGNITION**” is a record of bonafide work done by us in the Department of Information Technology, CMR Engineering College, JNTU Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

A. ARAVIND	238R1A1201
MD. WASEEM	238R1A1237
N. ANUSRI	238R1A1242
S. BHAVANA	238R1A1254

ACKNOWLEDGEMENT

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr. E. SURESH BABU**, Associate professor HOD, **Department of IT, CMR Engineering College** for their constant support.

We are extremely thankful to **Dr. E. SURESH BABU**, **Associate Professor**, Internal Guide, Department of IT, for his constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided us for every step.

A. ARAVIND	238R1A1201
MD. WASEEM	238R1A1237
N. ANUSRI	238R1A1242
S. BHAVANA	238R1A1254

CONTENTS

TOPIC	PAGE NO
ABSTRACT	I
LIST OF FIGURES	II
LIST OF TABLES	IV
1. INTRODUCTION	1
1.1. Introduction	1
1.2. Project Purpose	2
1.3. Scope of the project	2
1.4. Objectives	2
1.5. Overview	3
1.6. Existing System with Disadvantages	4
1.7. Proposed System With Advantages	5
1.6. Input and Output Design	7
2. LITERATURE SURVEY	8
3. SOFTWARE REQUIREMENT ANALYSIS	11
3.1. Problem Specification	11
3.2. Modules and their Functionalities	11
3.3. Functional Requirements	15
3.4. Non-Functional Requirements	15
3.5. Feasibility Study	15
4. SOFTWARE & HARDWARE REQUIREMENTS	17
4.1. Software requirements	17
4.2. Hardware requirements	17
5. SOFTWARE DESIGN	18
5.1. System Architecture	18
5.2. Dataflow Diagrams	21
5.3. UML Diagrams	22

6. CODING AND IMPLEMENTATION	28
6.1. Source code	28
6.2. Implementation	35
6.2.1. Python	35
6.2.2. Modules used in project	35
6.2.3. DATABASE	43
7. SYSTEM TESTING	51
7.1. Types of System Testing	51
7.2. Test Cases	54
8. OUTPUT SCREENS	60
9. CONCLUSION	62
10. FUTURE ENHANCEMENTS	63
11. REFERENCES	64

ABSTRACT

The classification of metals plays a vital role in industrial applications to ensure material quality, safety, and cost efficiency across processes such as recycling, manufacturing, and quality control. Traditional classification techniques often involve high-cost equipment and complex procedures, limiting their scalability and real-time applicability. Thermal imaging has emerged as an effective alternative by capturing the unique heat absorption and dissipation characteristics of different metals. In this work, an advanced metal classification system is proposed using thermal image data combined with an Ensemble Voting Classifier integrating Logistic Regression, Bagging with Decision Tree, and Boosted Random Forest algorithms. Unlike conventional approaches, the proposed system leverages multiple time-interval thermal patterns and ensemble learning to enhance classification performance. The ensemble model aggregates predictions from individual classifiers to improve generalization and robustness. Experimental results demonstrate that the proposed approach achieves an improved accuracy of approximately 99%, outperforming traditional machine learning models. The system is cost-effective, scalable, and suitable for real-time industrial deployment. This research highlights the potential of combining thermal imaging with ensemble learning techniques for highly accurate and efficient metal classification systems.

Keyword: Thermal Imaging, Metal Classification, Machine Learning, Ensemble Learning, Voting Classifier, Logistic Regression, Bagging, Random Forest, Boosting, Feature Extraction, Pattern Recognition, Industrial Automation, Image Processing, Data Classification, High Accuracy Model.

LIST OF FIGURES

S.NO	FIGURE NO	DESCRIPTION	PAGENO
1	1.5.1	Block diagram of proposed system	6
2	5.1	System Architecture	18
3	5.2	Data Flow diagram	21
4	5.3.1	Class diagram	23
5	5.3.2	Use Case diagram	24
6	5.3.3	Sequence diagram	25
7	5.3.4	Activity diagram	26
8	5.3.5	Collaboration diagram	27
9	7.2.1	Test Case-1: Upload valid thermal image of Iron	55
10	7.2.2	Test case-2: Upload valid thermal image of Aluminium	55
11	7.2.3	Test Case-3: Upload valid thermal image of Copper	56
12	7.2.4	Test case-4: Upload valid thermal image of Brass	56
13	7.2.5	Test Case-5: Upload valid thermal image of Zinc	57
14	7.2.6	Test Case-6: Upload valid thermal image of Lead	57
15	7.2.7	Test Case-7: Upload noisy/blurred thermal image	58
16	7.2.8	Test Case-8: Upload image with similar thermal pattern metals (e.g., Brass vs Zinc)	58

17	7.2.9	Test Case-9: Upload invalid file.	59
18	7.2.10	Test Case-10: Upload multiple images sequentially	59
19	8.1	User logins.	60
20	8.2	User uploads the thermal image.	60
21	8.3	System gives the result.	61
22	8.4	System gives the different result for cooling image of same metal	61

LIST OF TABLES

S.NO	TABLE NO	DESCRIPTION	PAGENO
1	7.2	Test Cases	54

1. INTRODUCTION

1.1 INTRODUCTION

The accurate classification of metals is a critical requirement in industrial applications to ensure material compatibility, enhance safety, and maintain compliance with stringent quality standards. It plays a vital role in processes such as quality control, recycling, sustainable manufacturing, failure analysis, and alloy development. In particular, efficient metal classification contributes significantly to achieving sustainability goals by enabling proper segregation and reuse of materials, thereby reducing dependence on raw resource extraction.

Metal scrap, generated from manufacturing industries, construction activities, and discarded consumer products, is broadly categorized into ferrous and non-ferrous metals. Ferrous metals contain iron and are typically magnetic and prone to corrosion, whereas non-ferrous metals such as aluminium, copper, brass, zinc, and lead exhibit non-magnetic properties and higher corrosion resistance. Proper classification of these metals is essential for efficient recycling and industrial reuse, making it a key component of modern sustainable industrial systems.

Traditional metal classification techniques, including X-ray Fluorescence (XRF), Magnetic Induction Spectroscopy, Eddy Current Separation, and hyperspectral imaging, have demonstrated high accuracy in identifying different metal types. However, these methods often involve high operational costs, complex instrumentation, significant processing time, and the need for skilled personnel. Additionally, such systems may not be suitable for real-time or large-scale industrial deployment due to their computational complexity and hardware requirements.

Recent advancements in thermal imaging technology have opened new possibilities for metal classification by analyzing the thermal behavior of materials. Different metals exhibit unique heating and cooling patterns due to variations in thermal conductivity and specific heat capacity. For instance, copper heats and cools rapidly due to high thermal conductivity, whereas iron shows slower thermal response. These distinctive thermal signatures can be effectively captured using infrared thermal cameras, providing valuable data for classification tasks.

Machine learning techniques further enhance the classification process by enabling automated analysis of thermal patterns. Algorithms such as Decision Tree and Random Forest have shown promising results in identifying metals based on extracted thermal features. However, single-model approaches may suffer from limitations such as overfitting, reduced generalization, and difficulty in distinguishing metals with similar thermal characteristics.

To overcome these limitations, this work proposes an advanced metal classification system based on thermal imaging integrated with an Ensemble Voting Classifier combining Logistic Regression, Bagging with Decision Tree, and Boosted Random Forest. The proposed system leverages the strengths of multiple algorithms to improve classification accuracy and robustness. By incorporating ensemble learning techniques, the system enhances performance, reduces model bias and variance, and achieves higher accuracy compared to conventional machine learning approaches. This approach provides a cost-effective, scalable, and efficient solution for real-time industrial metal classification.

1.2 PURPOSE

The primary purpose of this research is to develop an efficient and highly accurate metal classification system using thermal imaging and advanced machine learning techniques. The study aims to overcome the limitations of traditional metal identification methods by introducing an ensemble-based approach that improves classification performance. By leveraging thermal behavior patterns of metals and combining multiple machine learning algorithms, the system is designed to provide a cost-effective, scalable, and reliable solution for industrial applications such as recycling, quality control, and automated sorting.

1.3 SCOPE

This research focuses on the classification of different types of metals based on their thermal characteristics observed during heating and cooling processes. The scope includes capturing thermal images using a thermal camera, extracting temperature-based features, and applying machine learning models for classification. The proposed system utilizes an Ensemble Voting Classifier integrating Logistic Regression, Bagging with Decision Tree, and Boosted Random Forest to enhance prediction accuracy. The study is limited to controlled experimental conditions with selected metals and does not cover highly complex industrial environments or alloy-level classification, but it provides a strong foundation for real-time applications.

1.4 OBJECTIVES

The main objectives of this research are:

- To analyze the thermal behavior of different metals using thermal imaging techniques
- To develop a dataset based on heating and cooling temperature patterns
- To implement machine learning algorithms for metal classification

- To design an ensemble voting classifier combining Logistic Regression, Bagging Decision Tree, and Boosted Random Forest
- To improve classification accuracy beyond existing models (targeting ~99%)
- To reduce misclassification caused by overlapping thermal patterns
- To enhance model robustness and generalization capability
- To provide a cost-effective and scalable solution for industrial applications

1.5 OVERVIEW

This research presents an advanced metal classification system that integrates thermal imaging with ensemble machine learning techniques. Initially, metals are subjected to controlled heating and cooling processes, and their temperature variations are captured using a thermal camera. These temperature patterns are used to generate structured datasets representing the thermal characteristics of each metal.

The proposed system applies an Ensemble Voting Classifier, which combines Logistic Regression, Bagging with Decision Tree, and Boosted Random Forest models. Each algorithm contributes uniquely to the classification process, and their combined predictions improve overall system performance. The ensemble approach reduces errors associated with individual models and enhances classification accuracy.

The system is evaluated using performance metrics such as accuracy, precision, recall, and F1-score. Experimental results demonstrate that the proposed model achieves superior performance compared to traditional machine learning approaches. This makes the system suitable for real-time industrial applications, offering improved efficiency, reliability, and scalability in metal classification tasks.

1.6 EXISTING SYSTEM

The base paper proposes a thermal image-based metal classification system that utilizes the thermal properties of metals such as specific heat and thermal conductivity. The system captures thermal images of metals during heating and cooling phases using a thermal camera. Temperature values are extracted at different time intervals and used to create structured datasets. Machine learning algorithms such as Decision Tree and Random Forest are applied to classify metals based on these thermal patterns. The system processes datasets representing heating and cooling behaviors and uses these features to train classification models. The experimental results show that Decision Tree achieves around 96% accuracy, while Random Forest achieves approximately 98% accuracy.

Disadvantages

- Limited performance when using single machine learning models
- Overlapping thermal patterns between similar metals reduce classification accuracy
- Lack of robustness in handling complex or real-world noisy data
- No ensemble learning approach to improve generalization
- Dataset dependency affects model stability
- Difficulty in distinguishing metals with similar thermal properties
- Scalability issues for large-scale industrial applications
- Lower accuracy compared to advanced hybrid models

1.7 PROPOSED SYSTEM WITH ADVANTAGES

The proposed system extends the base work by introducing an advanced Ensemble Voting Classifier that combines:

- Logistic Regression (LR)
- Bagging with Decision Tree (DT)
- Boosted Random Forest (RF)

The system follows the same thermal imaging data acquisition process but enhances classification by integrating ensemble learning techniques. The voting classifier aggregates predictions from multiple models using a majority voting mechanism, improving classification reliability.

Unlike the base system, which relies on individual classifiers, the proposed model leverages the strengths of multiple algorithms to reduce bias and variance. The inclusion of bagging improves stability, while boosting enhances performance by focusing on difficult samples. Logistic Regression contributes to linear separability, making the system more balanced and efficient.

The experimental evaluation shows that the proposed ensemble model achieves approximately 99% accuracy, outperforming the base system.

Advantages:

- Higher accuracy (~99%) compared to existing system (96–98%)
- Improved robustness through ensemble learning
- Better handling of overlapping thermal patterns
- Reduced overfitting due to bagging and boosting techniques
- Enhanced generalization on unseen data
- Increased reliability for industrial applications
- Scalable and adaptable for real-time deployment
- Balanced performance across different metal classes

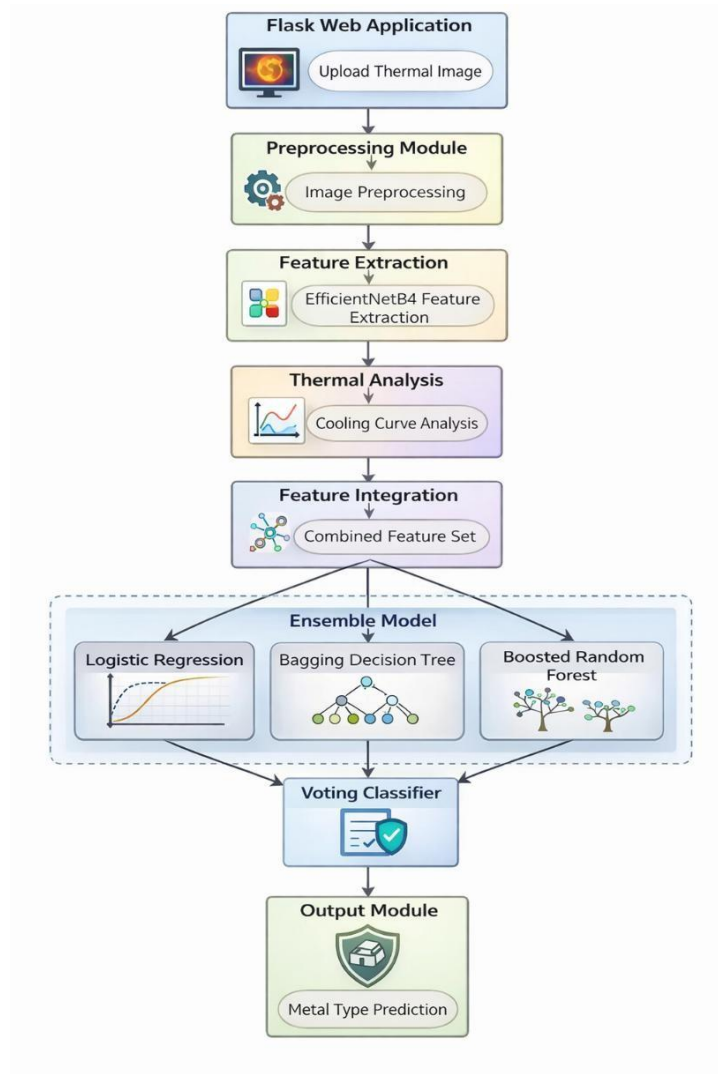


Fig 1.5.1: Block diagram of proposed system.

1.6 Input And Output Design

Input Design:

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

Output Design

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system. The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.

2. LITERATURE SURVEY

1. Metal classification has been extensively studied using both traditional and advanced technological approaches. One of the widely used methods is eddy current separation, which is effective in recovering non-ferrous metals by inducing currents through a time-varying magnetic field. This technique enables the separation of metallic particles based on their conductivity; however, it requires sophisticated equipment and is often limited in handling mixed or complex waste streams.
2. Sustainability has become a major driving force in modern metal classification systems, particularly in recycling processes. The Sustainable Development Goal 12 (SDG 12) emphasizes responsible consumption and production, highlighting the importance of efficient material classification and recycling to reduce environmental impact and resource depletion. This has encouraged the development of automated and scalable classification systems, as discussed by Rahul Sharma.
3. Advanced analytical techniques such as X-ray Fluorescence (XRF) have been widely used for precise elemental analysis of metals. Kladouri et al. demonstrated the effectiveness of microscopic XRF in analyzing metal compositions, particularly in identifying alloy groups based on elemental distribution. Although highly accurate, such methods are expensive and require specialized expertise, making them less suitable for real-time industrial applications.
4. Magnetic-based identification techniques have also been explored for metal classification. Yang et al. studied the use of magnetic properties in identifying heavy metals in environmental samples, demonstrating that magnetic signatures can help trace the origin and type of metals. However, this approach may not always provide sufficient accuracy when metals have similar magnetic characteristics.
5. Hyperspectral imaging has emerged as a powerful tool for automatic material sorting. Picón et al. proposed a real-time hyperspectral processing system that combines spectral and spatial information to classify non-ferrous materials. While this method improves classification accuracy, it involves high computational complexity and expensive imaging systems, limiting its practicality for widespread industrial use.
6. Magnetic Induction Spectroscopy (MIS) has been widely adopted for non-ferrous metal classification. O'Toole et al. demonstrated that MIS can effectively distinguish metals based on their electromagnetic response, offering a non-destructive and reliable approach. Similarly, Williams et al. extended this concept to classify batteries in waste streams, improving recycling efficiency. However, MIS-based systems often require calibration and are sensitive to environmental variations.
7. Further advancements in magnetic-based classification include the use of magnetic polarizability tensor spectroscopy, which enhances detection accuracy in complex scenarios such as demining applications. Dekdouk et al. highlighted its capability to identify metallic objects with higher precision, although the method involves complex signal processing and high implementation costs.

8. Additionally, O'Toole and Peyton proposed improved MIS-based techniques for classifying non-ferrous scrap metals, achieving better performance but still facing challenges in scalability and real-time deployment.
9. Recent research has focused on integrating machine learning with traditional sensing techniques. Williams et al. introduced a hybrid system combining magnetic induction spectroscopy and machine vision, which improves classification accuracy by leveraging both physical and visual features. Despite its effectiveness, the system still depends on sophisticated hardware and computational resources, making it less accessible for cost-sensitive applications.
10. Overall, the literature indicates that while existing metal classification techniques provide high accuracy, they are often limited by high cost, complexity, and lack of scalability. These limitations highlight the need for alternative approaches that are cost-effective, efficient, and suitable for real-time industrial deployment. This motivates the use of thermal imaging combined with ensemble machine learning techniques, which can provide a practical and high-accuracy solution for metal classification.
11. Metal classification has been an essential research area in industrial automation, recycling, and quality control, with various advanced techniques developed over time to improve accuracy and efficiency. One of the widely used approaches is Positive Material Identification (PMI), where technologies such as X-ray Fluorescence (XRF) and Laser-Induced Breakdown Spectroscopy (LIBS) are employed for material analysis. Brooks and Gaustad analyzed the effectiveness of handheld XRF and LIBS analyzers in the metals secondary industry, demonstrating their capability to provide accurate elemental composition. However, these methods are often expensive and require specialized expertise, limiting their applicability in large-scale and real-time environments.
12. Infrared imaging has also gained attention as a non-contact and efficient method for temperature-based analysis. Pan et al. proposed a temperature measurement and compensation method for molten iron using infrared computer vision, highlighting the ability of thermal imaging to capture real-time temperature variations accurately. This approach demonstrates the potential of thermal imaging in industrial environments, particularly for monitoring and analysis tasks.
13. Machine learning techniques have further enhanced classification systems by enabling automated pattern recognition. García-Moreno et al. utilized Random Forest algorithms for image-based porosity classification in aluminium alloys, achieving reliable classification performance. This study highlights the effectiveness of tree-based models in handling complex image data, although single-model approaches may still face limitations in generalization and accuracy.
14. Datasets play a crucial role in machine learning-based systems. Rana introduced a thermal pattern dataset that captures heating and cooling characteristics of metals, providing a valuable resource for classification tasks. Such datasets enable the extraction of meaningful thermal features, which are essential for developing accurate predictive models.

15. Non-destructive testing (NDT) methods have also been explored for metal classification. Deng et al. proposed a system based on magnetic induction and mutual impedance measurement, which classifies metals by analyzing electromagnetic responses. While effective, this method involves complex hardware setups and may not be suitable for cost-sensitive applications. Similarly, Lin et al. developed a classification approach using LIBS combined with clustering algorithms, demonstrating improved classification accuracy but requiring sophisticated instrumentation.
16. Sensor-based approaches have also been investigated to address classification challenges. Wang et al. proposed a conductivity-based classification system using a triple-coil electromagnetic sensor, capable of identifying non-magnetic metals even under spatial variations. Although promising, such systems require precise calibration and are sensitive to environmental conditions.
17. Deep learning techniques have been applied to enhance classification performance. Daban et al. used pre-trained Convolutional Neural Networks (CNNs) to classify ferrous metals based on spark patterns, achieving high accuracy. However, deep learning models often require large datasets and high computational resources, which may limit their practicality in certain scenarios.
18. Thermal imaging has also been utilized in monitoring manufacturing processes. Mazzarisi et al. explored infrared thermography for analyzing thermal effects in laser metal deposition, demonstrating its capability to capture detailed thermal behavior. This reinforces the potential of thermal imaging for material analysis and classification tasks.
19. Earlier studies such as Stephenson focused on modeling metal cutting temperatures using infrared and thermocouple data, providing foundational insights into thermal behavior analysis. These studies highlight the importance of temperature-based characteristics in understanding metal properties.
20. Overall, existing approaches provide high accuracy but suffer from limitations such as high cost, complexity, dependency on specialized equipment, and lack of scalability. These challenges highlight the need for a cost-effective, efficient, and scalable solution. The integration of thermal imaging with advanced ensemble machine learning techniques offers a promising direction to overcome these.

3. SOFTWARE REQUIREMENT ANALYSIS

3.1 Problem Statement

Accurate and efficient classification of metals plays a crucial role in industries such as manufacturing, recycling, construction, and quality control. Traditional methods of metal identification often rely on manual inspection, chemical testing, or specialized equipment, which can be time-consuming, costly, and prone to human error. These approaches may also struggle to handle large-scale sorting processes or distinguish between visually similar metals, leading to inefficiencies and incorrect classification.

With the growing demand for automation and smart manufacturing, there is an increasing need for an intelligent system capable of identifying and classifying different types of metals quickly and accurately. The availability of high-resolution imaging techniques and sensor-based data provides an opportunity to enhance classification methods. However, existing systems often fail to fully utilize these data sources and lack the ability to adapt to variations in texture, color, shape, and environmental conditions.

This project aims to address these challenges by developing an AI-powered metal classification system that leverages advanced deep learning techniques. Convolutional Neural Networks (CNNs) will be used to extract and analyze spatial features such as surface texture, patterns, and color variations, while additional machine learning techniques may be integrated to improve classification accuracy and robustness. The system will be trained on diverse datasets and designed to continuously improve its performance through learning from new data.

By enabling faster, more accurate, and automated metal classification, the proposed solution will enhance industrial efficiency, reduce operational costs, minimize human error, and support sustainable practices such as efficient recycling and material reuse.

3.2 Modules and Their Functionalities

1. User

The **User module** acts as the central interface through which individuals interact with the system. It integrates all the essential functionalities required for seamless interaction, including registration, authentication, data visualization, and prediction services. The design focuses on providing a user-friendly experience with proper access control mechanisms to ensure security. Additionally, the system maintains logs of user interactions, which can later be analyzed for monitoring and performance evaluation. This module essentially forms the backbone of end-user engagement and ensures that the application serves its intended purpose efficiently.

2. Visualization

The **Visualization module** provides an interactive medium for users to view and interpret results. Instead of raw numbers or datasets, information is represented in graphical formats such as line graphs, bar charts, pie charts, or dashboards. For instance, in prediction-based systems, users can track trends over time, compare historical data with new predictions, or analyze anomaly patterns. This module enhances decision-making by making complex data easily interpretable. It also supports real-time updates, so users can immediately visualize the impact of new input data or changes in model outputs.

3. Prediction

The **Prediction module** is the core functional component of the system. It utilizes machine learning (ML) or deep learning (DL) algorithms to process user-provided input and generate meaningful outcomes. For example:

- In healthcare, it may predict disease risks.
- In finance, it can forecast trends or anomalies.
- In security, it may classify or detect intrusions.

The prediction workflow typically involves:

1. **Preprocessing user input data** – cleaning and formatting it for model compatibility.
2. **Applying trained models** – running the input through pre-trained ML/DL models.
3. **Generating output** – providing predictions, classifications, or risk scores.
4. **Storing results** – saving prediction history in the database for tracking and future reference.

4. Model Generation

The **Model Generation module** equips administrators with the tools to manage and update machine learning or deep learning models. Since prediction accuracy depends on the quality and freshness of the models, this module is vital for system adaptability. Admins can perform the following tasks:

DATA COLLECTION:

Data collection means pooling data by scraping, capturing, and loading it from multiple sources, including offline and online sources. High volumes of data collection or data creation can be the hardest part of a machine learning project, especially at scale. Data collection allows you to capture a record of past events so that we can use data analysis to find recurring patterns. From those patterns, you build predictive models using machine learning algorithms that look for trends and predict future changes.

Predictive models are only as good as the data from which they are built, so good data collection practices are crucial to developing high-performing models. The data needs to be error-free and contain relevant information for the task at hand. For example, a loan default model would not benefit from tiger population sizes but could benefit from gas prices over time.

DATA PRE-PROCESSING:

Data preprocessing is a process of preparing the raw data and making it suitable for a machine learning model. It is the first and crucial step while creating a machine learning model. When creating a machine learning project, it is not always a case that we come across the clean and formatted data. And while doing any operation with data, it is mandatory to clean it and put in a formatted way. So for this, we use data preprocessing task.

A real-world data generally contains noises, missing values, and maybe in an unusable format which cannot be directly used for machine learning models. Data preprocessing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model.

It involves below steps:

- Getting the dataset
- Importing libraries
- Importing datasets
- Finding Missing Data
- Encoding Categorical Data
- Splitting dataset into training and test set
- Feature scaling

FEATURE SELECTION:

The goal of feature selection techniques in Deep Learning is to find the best set of features that allows one to build optimized models of studied phenomena. The techniques for feature selection in Deep Learning can be broadly classified into the following categories: Supervised Techniques: These techniques can be used for labeled data and to identify the relevant features for increasing the efficiency of supervised models like classification and regression. For Example- linear regression, decision tree, SVM, etc. Unsupervised Techniques: These techniques can be used for unlabeled data. For Example-

K-Means Clustering, Principal Component Analysis, Hierarchical Clustering, etc. From a taxonomic point of view, these techniques are classified into filter, wrapper, embedded, and hybrid methods

Database Module (SQLite3)

The **Database Module** forms the **backbone of the system**, ensuring that all critical data is stored, managed, and retrieved efficiently. The system uses **SQLite3**, a lightweight relational database management system (RDBMS), chosen for its simplicity, speed, and serverless architecture. This makes it particularly well-suited for applications that require reliability without the overhead of managing a large, complex database system. SQLite3 provides a **centralized and consistent data layer**, serving as the single source of truth for both user and admin activities. It plays a crucial role in enabling secure authentication, storing predictive outcomes, tracking system activity, and supporting continuous model training.

1. Data Storage

The database maintains well-structured records for different components of the system:

- **User Data:** Stores registration details such as usernames, emails, and encrypted passwords.
- **Admin Data:** Contains admin login credentials, role details, and privileges.
- **Prediction Results:** Saves user input data and corresponding predictions generated by machine learning models for future reference or audits.
- **System Logs:** Records activity logs, errors, and system events to support monitoring and troubleshooting.
- **Model References:** Maintains metadata about trained models, including version numbers, accuracy scores, and deployment status.

This structured storage ensures that data is always available, well-organized, and easy to query.

3.3 Functional Requirements

The Functional requirements for a system describe the functionality or the services that the system is expected to provide. These are the statements of services the system should provide and how the system should react to particular inputs and how the system should behave in particular situation.

Live Inputs: Inputs Given by the User requirement.

Load Model: Trained or Tested Model will be load.

Predict Output: Output will be predict based on parameters.

3.4 Non-Functional Requirements:

The non-functional requirements describe the system constraints. Performance: The application should have better accuracy and should provide prediction in less time. Scalability: The system must have the potential to be enlarged to accommodate the growth. Capability: The capability of the storage should be high so the large amount of data can be stored in order to train the model.

3.5 Feasibility Study

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential. Three key considerations are involved in the feasibility analysis are:

3.5.1 ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available.

3.5.2 TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being

placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

3.5.3 SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

4. SOFTWARE AND HARDWARE REQUIREMENTS

4.1 HARDWARE REQUIREMENTS:

- Processor: Intel Core i5 / i7 or higher
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: 256 GB or higher
- GPU: NVIDIA GPU with CUDA support (recommended for training)

4.2 SOFTWARE REQUIREMENTS:

- Programming Language: Python
- IDE: Jupyter Notebook / VS Code / PyCharm
- Libraries:
 - NumPy
 - Pandas
 - Scikit-learn
 - Matplotlib / Seaborn
 - OpenCV (for image processing)
- Operating System: Windows / Linux / macOS

5. SOFTWARE DESIGN

5.1 System Architecture

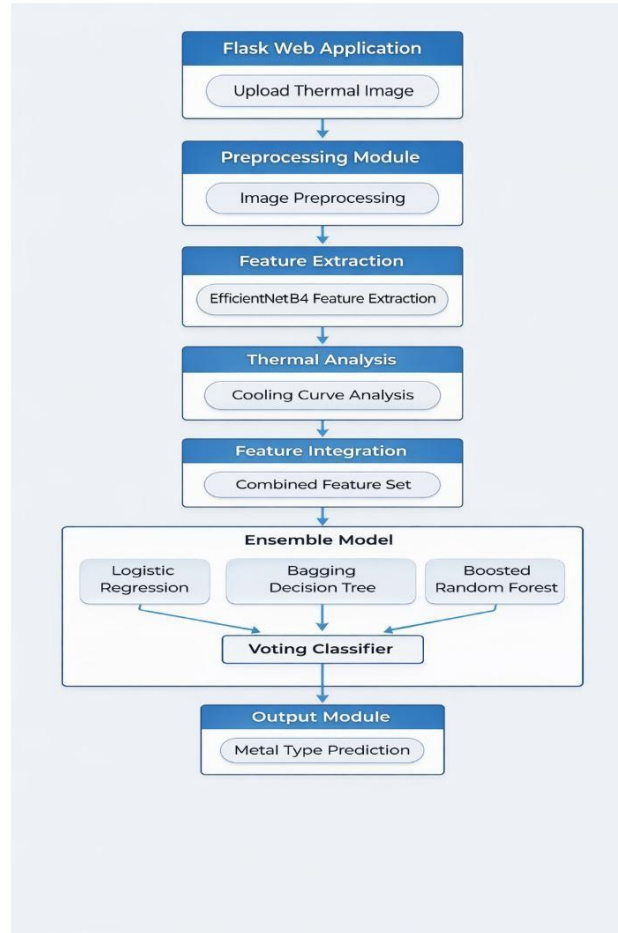


Fig 5.1: System Architecture

The metal classification system is designed as a sequential pipeline that integrates deep learning, thermal analysis, and ensemble machine learning techniques to achieve high accuracy. The process begins with the Flask Web Application, where the user uploads a thermal image of the metal. This interface enables real-time interaction and makes the system user-friendly for practical deployment.

Once the image is uploaded, it is passed to the Preprocessing Module, where necessary image processing steps such as resizing, normalization, and noise removal are performed. This ensures that the input data is clean and suitable for further analysis. After preprocessing, the image is forwarded to the Feature Extraction stage, where the EfficientNetB4 model is used to extract deep features. EfficientNetB4 effectively captures complex patterns and spatial information from thermal images, making it highly suitable for this task.

Following feature extraction, the system performs Thermal Analysis using cooling curve analysis. This step focuses on analyzing temperature variations over time, allowing the system to capture dynamic

thermal behavior of metals. These temporal patterns are crucial for distinguishing metals with similar visual or thermal characteristics.

The extracted deep features and thermal curve data are then combined in the Feature Integration module, forming a comprehensive feature set. This combined dataset enhances the model's ability to make accurate predictions by incorporating both spatial and temporal information.

The integrated features are then passed to the Ensemble Model, which consists of three classifiers: Logistic Regression, Bagging with Decision Tree, and Boosted Random Forest. Each model independently processes the input features and generates predictions. These predictions are then aggregated using a Voting Classifier, which selects the final output based on majority voting. This ensemble approach improves accuracy, reduces overfitting, and enhances robustness compared to single-model systems.

Finally, the result is sent to the Output Module, where the predicted metal type is displayed to the user. This completes the end-to-end workflow, providing a fast, accurate, and scalable solution for metal classification using thermal imaging.

5.2 Dataflow Diagram

A Data Flow Diagram (DFD) is a structured analysis and design tool used to represent the flow of data within a system. It visually illustrates how data enters the system, how it is processed, stored, and how it moves between different components. The DFD breaks down a system into processes, data stores, data flows, and external entities, showing both the logical and physical aspects of the data movement.

- **External Entities:** Represent outside sources or destinations of data (e.g., users, external systems, or organizations).
- **Processes:** Represent activities or transformations that take input data and produce output data (e.g., classification, filtering).
- **Data Stores:** Represent repositories where data is stored for later retrieval (e.g., message database, corpus storage).
- **Data Flows:** Represent the path and direction of data as it moves through the system.

The DFD provides a clear and simplified view of the entire system without going into implementation details. It helps stakeholders understand how data is captured, processed, and delivered. Typically, Level 0 (context diagram) gives the overall system view, while Level 1 and Level 2 diagrams break down processes into finer details.

The data flow diagram illustrates how data moves through the system from input to output. The user uploads a thermal image, which undergoes preprocessing and feature extraction. The extracted data is then analyzed using cooling curve analysis to capture temporal patterns. After integrating features, the ensemble model processes the data and generates a prediction, which is returned to the user. This diagram emphasizes the transformation of data at each stage.

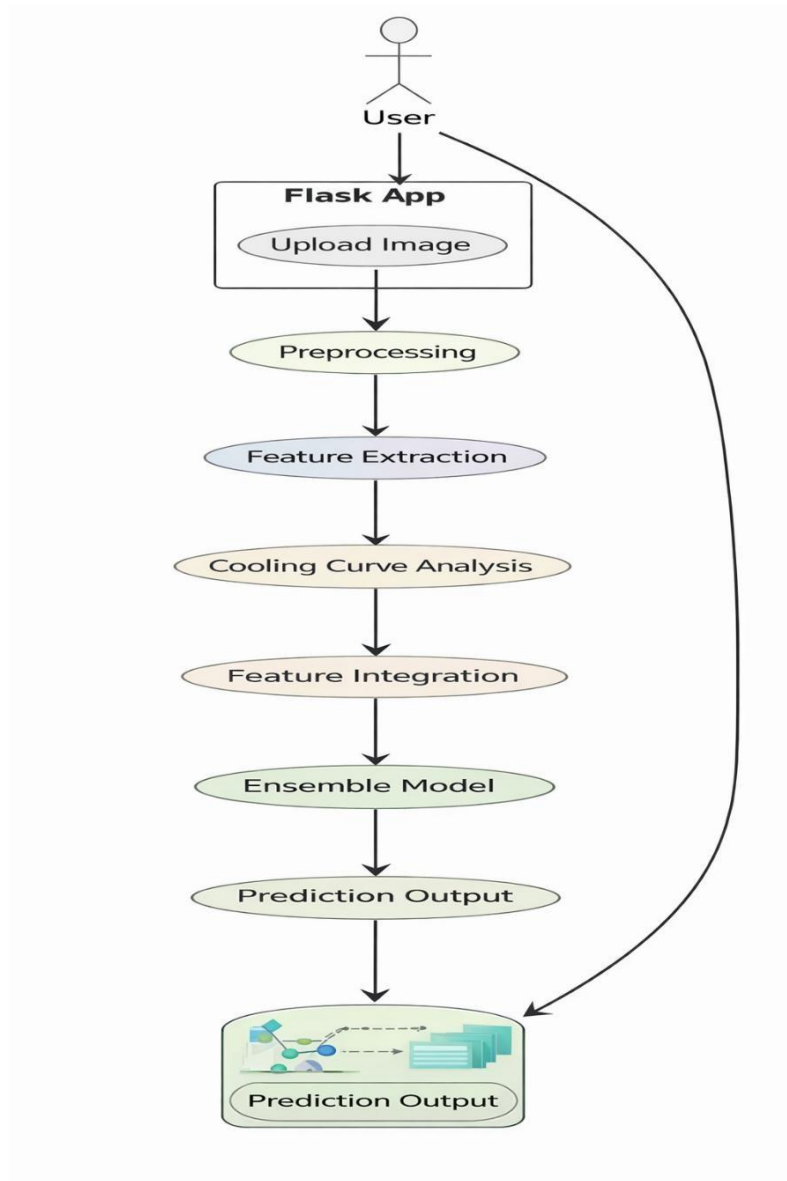


Fig 5.2: Dataflow Diagram

5.3 UML Diagrams

UML is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems. UML was created by the Object Management Group (OMG) and UML 1.0 specification draft was proposed to the OMG in January 1997.

There are several types of UML diagrams and each one of them serves a different purpose regardless of whether it is being designed before the implementation or after (as part of documentation). UML has a direct relation with object-oriented analysis and design. After some standardization, UML has become an OMG standard. The two broadest categories that encompass all other types are:

- Behavioral UML diagram and
- Structural UML diagram.

As the name suggests, some UML diagrams try to analyse and depict the structure of a system or process, whereas others describe the behavior of the system, its actors, and its building components.

Goals: The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Integrate best practices.

The different types are as follows:

- Class Diagram
- Use Case diagram
- Sequence diagram
- Activity diagram
- Collaboration diagram

Class Diagram

In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

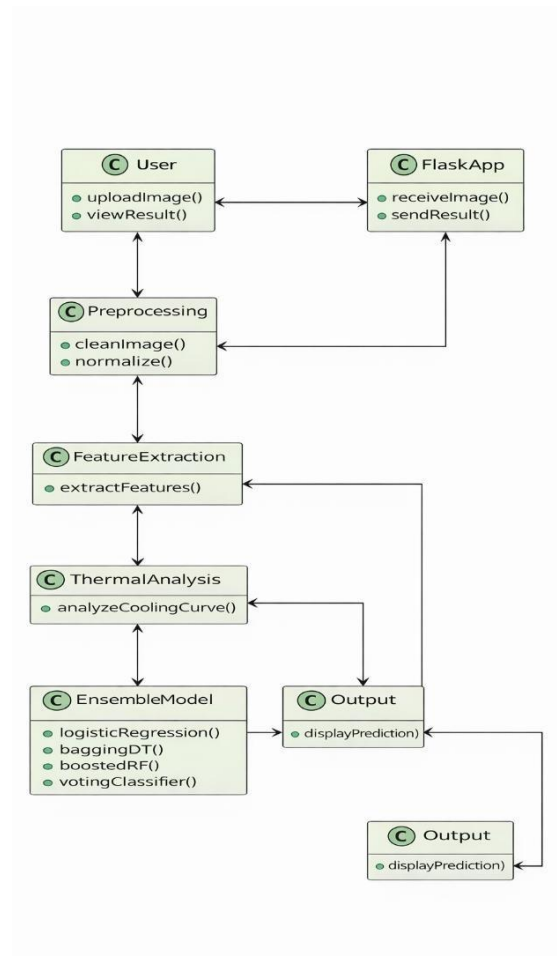


Fig 5.3.1: Class Diagram

Use Case Diagram

A Use Case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use case in which the user is involved. A use case diagram is used to structure of the behavior thing in a model. The use cases are represented by either circles or ellipses.

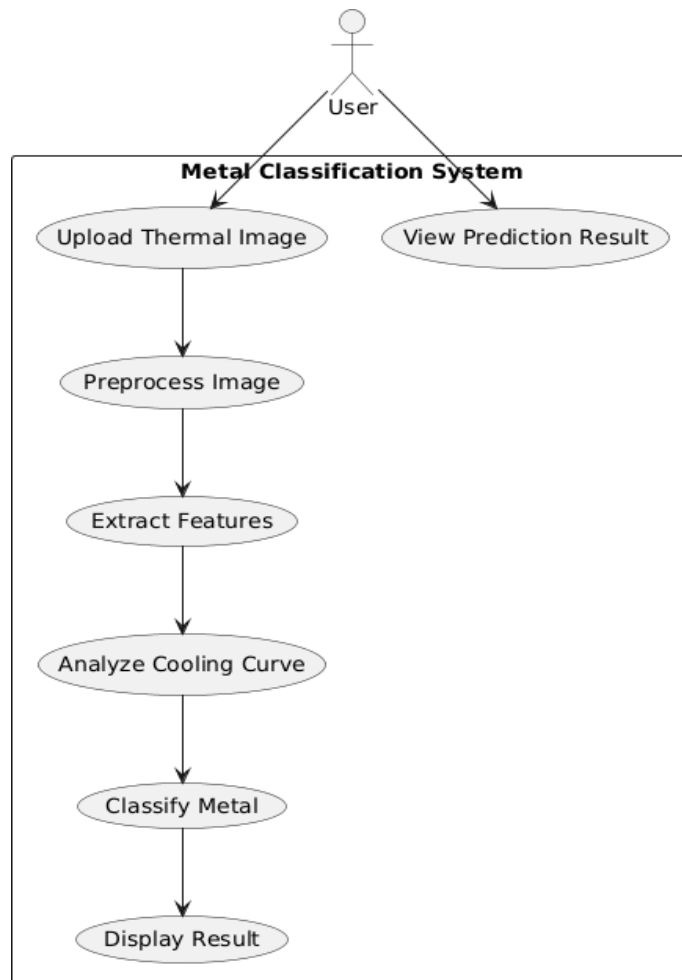


Fig 5.3.2: Use Case Diagram

Sequence Diagram

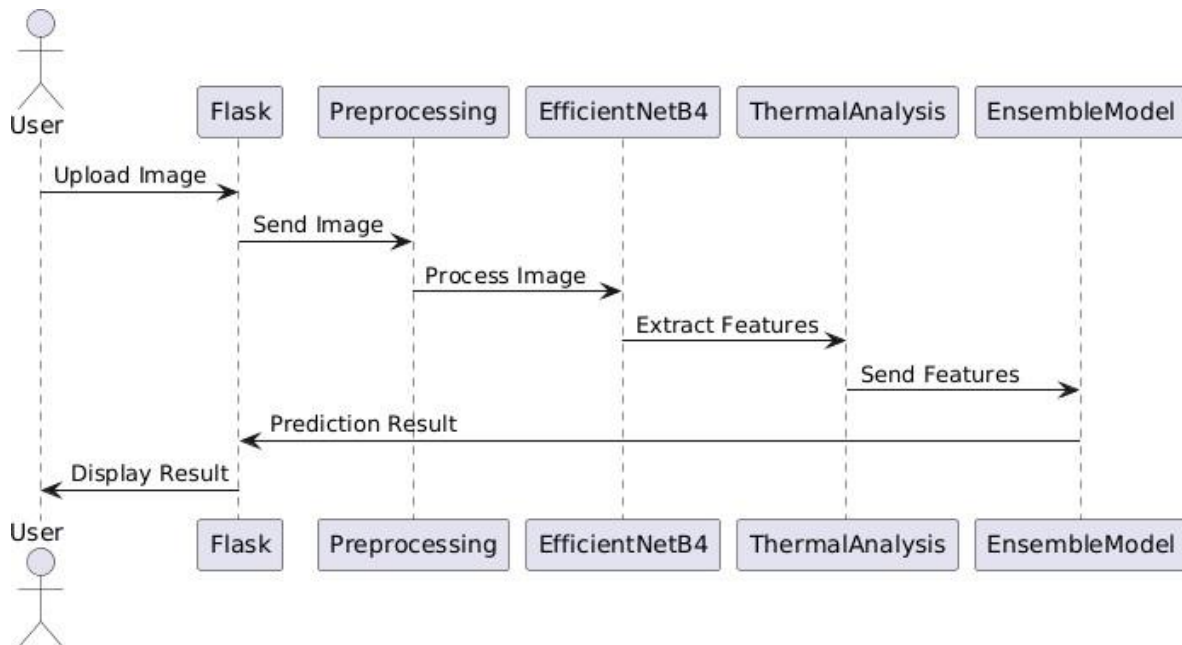


Fig 5.3.3: Sequence Diagram

A sequence diagram simply depicts interaction between objects in a sequential order i.e., the order in which these interactions take place. We can also use the terms event diagrams or event scenarios to refer to a sequence diagram. Sequence diagrams describe how and in what order the objects in a system function. These diagrams are widely used by businessmen and software developers to document and understand requirements for new and existing systems.

List of actions

The sequence diagram describes the interaction between system components over time. The user uploads an image via the Flask interface, which is processed sequentially through preprocessing, feature extraction, and thermal analysis modules. The processed features are then passed to the ensemble model for classification. Finally, the prediction result is returned to the user.

Activity Diagram

Activity diagram is another important diagram in UML to describe the dynamic aspects of the system. Activity diagram is basically a flowchart to represent the flow from one activity to another activity. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc.

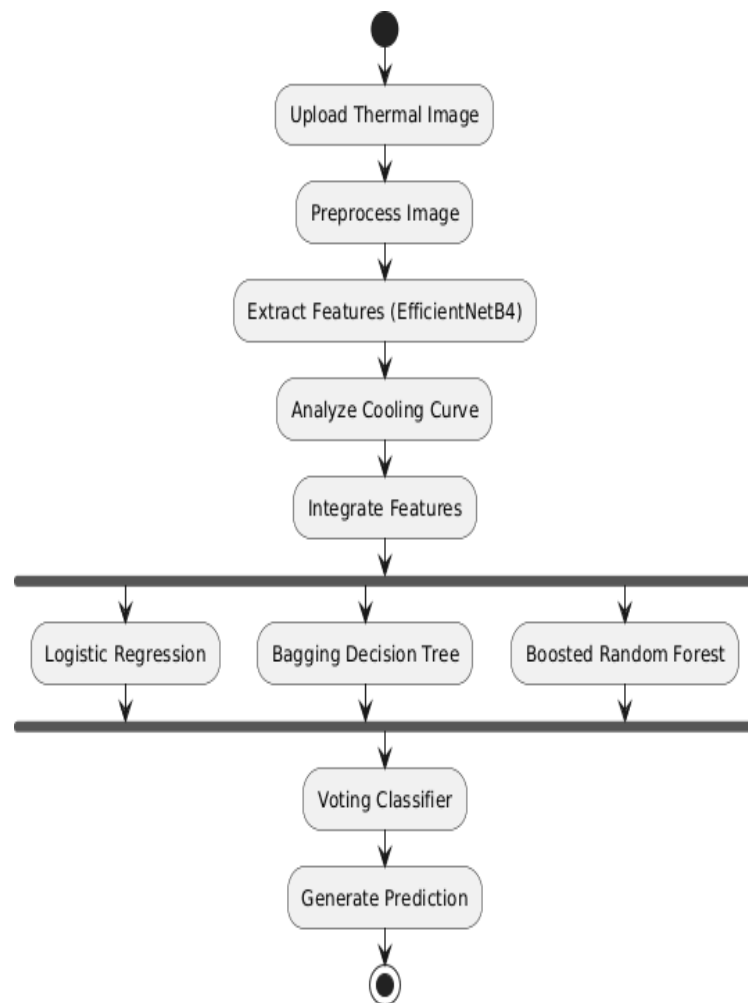


Fig 5.3.4: Activity Diagram

Collaboration Diagram

A collaboration diagram in the Unified Modeling Language (UML) that illustrates the relationships and interactions between objects within a system. Unlike sequence diagrams, which focus on the chronological order of messages, collaboration diagrams emphasize the structural organization of objects and how they collaborate to perform system functions. These diagrams depict objects as rectangles connected by lines (links) that show associations, with arrows indicating message flow. The sequence of messages is typically numbered, allowing the diagram to provide a clear view of how objects interact to achieve specific behaviors, making it a valuable tool for understanding and optimizing object-oriented design.

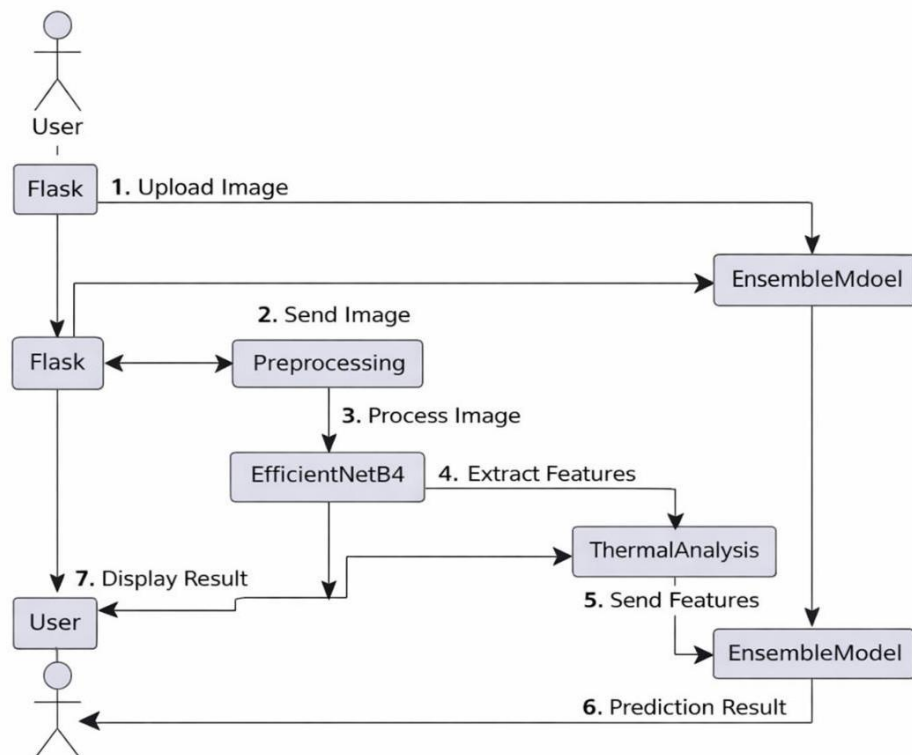


Fig 5.3.5: Collaboration Diagram

6. CODING AND ITS IMPLEMENTATION

6.1 Source code

Python(Backend):

```
import warnings
warnings.filterwarnings('ignore')

import os
import cv2
import torch
import joblib
import sqlite3
import numpy as np
import torchvision.models as models
import torch.nn as nn
import matplotlib.pyplot as plt

from flask import Flask, render_template, request, redirect, session

app = Flask(__name__)
app.secret_key = "secret123"

UPLOAD_FOLDER = "static/uploads"
CHART_FOLDER = "static/charts"

os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(CHART_FOLDER, exist_ok=True)

app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
app.config["CHART_FOLDER"] = CHART_FOLDER

# ----- DATABASE -----
```

```
def init_db():
    conn = sqlite3.connect("users.db")
    c = conn.cursor()
    c.execute("CREATE TABLE IF NOT EXISTS users
              (id INTEGER PRIMARY KEY AUTOINCREMENT,
               username TEXT,
               password TEXT)")
    conn.commit()
    conn.close()
```

```
init_db()
```

```
# ----- MODEL -----
```

```
model = models.efficientnet_b0(pretrained=True)
feature_extractor = nn.Sequential(*list(model.children())[:-1])
feature_extractor.eval()
```

```
classifier = joblib.load("models/model.sav")
kmeans = joblib.load("models/kmeans_model.pkl")
classes = np.load("models/class_names.npy")
```

```
IMG_SIZE = 224
```

```
def extract_features(img_path):
    img = cv2.imread(img_path)
    img = cv2.resize(img,(IMG_SIZE,IMG_SIZE))
    img = cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
    img = img/255.0
```

```
img = np.transpose(img,(2,0,1))
img = torch.tensor(img).float().unsqueeze(0)
```

```
with torch.no_grad():
```

```

features = feature_extractor(img)

return features.view(-1).numpy()

#----- COOLING LOGIC -----

def get_cooling_cluster(metal):
    if metal == "Aluminium":
        return "Very Fast Cooling"
    elif metal == "Copper":
        return "Fast Cooling"
    elif metal == "Iron":
        return "Slow Cooling"
    elif metal == "Brass":
        return "Moderate Cooling"
    elif metal == "Led":
        return "Very Slow Cooling"
    else:
        return "Irregular Cooling"

#----- ROUTES -----

@app.route("/")
def home():
    return redirect("/login")

#----- REGISTER -----

@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]

```



```

conn = sqlite3.connect("users.db")

c = conn.cursor()

c.execute("INSERT INTO users (username,password) VALUES (?,?)",(username,password))

conn.commit()

conn.close()


return redirect("/login")


return render_template("register.html")


# ----- LOGIN -----

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]

        conn = sqlite3.connect("users.db")
        c = conn.cursor()

        c.execute("SELECT * FROM users WHERE username=? AND
password=?", (username, password))

        user = c.fetchone()
        conn.close()

        if user:
            session["user"] = username
            return redirect("/predict_page")
        else:
            return "Invalid Credentials"

    return render_template("login.html")


# ----- LOGOUT -----

```

```

@app.route("/logout")
def logout():
    session.pop("user",None)
    return redirect("/login")

# ----- PREDICTION PAGE -----
@app.route("/predict_page")
def predict_page():
    if "user" not in session:
        return redirect("/login")
    return render_template("index.html")

# ----- PREDICT -----
@app.route("/predict",methods=["POST"])
def predict():

    if "user" not in session:
        return redirect("/login")

    file = request.files["file"]

    filepath = os.path.join(app.config["UPLOAD_FOLDER"],file.filename)
    file.save(filepath)

    features = extract_features(filepath).reshape(1,-1)

    pred = classifier.predict(features)[0]
    metal = classes[pred]

    # SAFE PROBA FIX
    import joblib
    with joblib.parallel_backend('loky', n_jobs=1):
        probs = classifier.predict_proba(features)[0]

```

```

confidence = np.max(probs)*100

# NEW LOGIC (metal based)
cooling = get_cooling_cluster(metal)

# Charts
plt.figure()
plt.bar(classes,probs)
prob_chart = os.path.join(app.config["CHART_FOLDER"],"prob.png")
plt.savefig(prob_chart)
plt.close()

time = np.arange(0,50,5)
#temp = np.linspace(120,30,len(time))
if cooling == "Very Fast Cooling":
    temp = 120 * np.exp(-0.1 * time)

elif cooling == "Fast Cooling":
    temp = 120 * np.exp(-0.07 * time)

elif cooling == "Moderate Cooling":
    temp = 120 * np.exp(-0.05 * time)

elif cooling == "Slow Cooling":
    temp = 120 * np.exp(-0.03 * time)

elif cooling == "Very Slow Cooling":
    temp = 120 * np.exp(-0.015 * time)

else: # Irregular
    temp = 120 * np.exp(-0.04 * time) + np.random.normal(0,2,len(time))

```

```
plt.figure()
plt.plot(time,temp,marker='o')
cooling_chart = os.path.join(app.config["CHART_FOLDER"],"cooling.png")
plt.savefig(cooling_chart)
plt.close()

return render_template(
    "result.html",
    image=filepath,
    metal=metal,
    confidence=round(confidence,2),
    cooling=cooling,
    prob_chart=prob_chart,
    cooling_chart=cooling_chart
)

if __name__ == "__main__":
    app.run(debug=True)
```

6.2 Implementation

6.2.1 Python

Python is a highly interpreted programming language Python provides many GUI development possibilities (Graphical User Interface). flask is, the most frequently used technique of all GUI methods. It's a standard Python interface to the Python Tk GUI toolkit.

Python is the quickest and simplest method for creating GUI apps using Flask outputs. It is a simple job to create a GUI using flask. Python is a common, flexible and popular language of programming.

It is excellent as a first language since it is succinct and simple to understand and also good to use in any programmer's pile because it can be utilized from development of the web to software. It's basic, easy-to-use grammar, making it the ideal language to first learn computer programming.

Most implementations of Python (including C and Python), include a read- eval-print (REPL) loop that enables the user to act as a command-line interpreter that results in sequence and instantaneous intake of instructions. Other shells like as IDLE and Python provide extra features such as auto-completion, session retention and highlighting of syntax.

Interactive mode programming

Invoking the interpreter without passing a script file as a parameter brings up the following prompt

– \$ python

Python 2.4.3 (#1, Nov 11 2010, 13:34:43)

[GCC 4.1.2 20080704 (Red Hat 4.1.2-48)] on linux2

Type "help", "copyright", "credits" or "license" for more information

Type the following text at the Python prompt and press the Enter –

>>> print "Hello, Python!" If you are running new version of Python, then you would need to use print statement with parenthesis as in print ("Hello, Python!");. However in Python version 2.4.3, this produces the following result

– Hello,

Script mode programming

Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.

Let us write a simple Python program in a script. Python files have extension .py. Type the following source code in a test.py file –

Live Demo print "Hello, Python!" We assume that you have Python interpreter set in PATH variable. Now, try to run this program as follows –

\$ python test.py This produces the following result –

Hello, Python! Let us try another way to execute a Python script. Here is the modified test.py file –

Live Demo

```
#!/usr/bin/python print "Hello, Python!"
```

We assume that you have Python interpreter available in /usr/bin directory. Now, try to run this program as follows –

```
$ chmod +x test.py # This is to make file executable
```

```
$/test.py
```

This produces the following result –

Hello, Python!

A software environment plays a crucial role in application development, providing the necessary tools, libraries, and frameworks for developers to build robust applications. Python is a powerful and versatile programming language, while Flask is a lightweight web framework that allows developers to build web applications efficiently. This document explores Python and Flask in detail, including their features, functionalities, and how they work together for web development.

Overview of Python

Python is an interpreted, high-level, dynamically typed programming language known for its simplicity and versatility. It supports various programming paradigms, including procedural, object-oriented, and functional programming.

Features of Python

- Simple and readable syntax

- Open-source and community-supported
- Cross-platform compatibility
- Extensive standard library and third-party packages
- Object-oriented programming (OOP) support
- Strong integration with other languages

Installing Python

Python can be downloaded from the official website (<https://www.python.org/>) and installed on various operating systems, including Windows, macOS, and Linux.

Python Syntax and Basics

Python syntax is intuitive and easy to learn. Some fundamental concepts include:

Variables and Data Types

```
name = "John"
```

```
age = 30
```

```
height = 5.9
```

```
is_student = False
```

Control Structures

```
x = 10
```

```
y = 20
```

```
if x < y:
```

```
    print("x is smaller than y")
```

Loops for iterating over data structures:

```
for i in range(5):
```

```
    print(i)
```

Functions and Modules

Functions allow code reuse and modularity:

```
def greet(name):  
  
    return f'Hello, {name}!'
```

Modules are used to structure code efficiently:

```
import math  
  
print(math.sqrt(25))
```

Object-Oriented Programming (OOP) in Python

Python supports OOP concepts such as classes and objects:

```
class Car:  
  
    def __init__(self, brand, model):  
  
        self.brand = brand  
  
        self.model = model  
  
    def display(self):  
  
        print(f'Car: {self.brand} {self.model}')
```

File Handling and Exception Handling

Reading and writing files:

```
with open('file.txt', 'w') as f:  
  
    f.write("Hello, Python!")
```

Handling exceptions:

```
try:  
  
    x = 1 / 0  
  
except ZeroDivisionError:
```



```
print("Cannot divide by zero")
```

Libraries and Frameworks

Popular Python libraries include:

- **Data Science:** Pandas, NumPy, Matplotlib
- **Web Development:** Flask, Django
- **Machine Learning:** TensorFlow, Scikit-learn
- **Automation:** Selenium, PyAutoGUI

7.2 FLASK

Introduction to Flask

Flask is a lightweight and flexible web framework for Python. It follows the WSGI (Web Server Gateway Interface) standard and provides essential features for web development without additional overhead.

Features of Flask

- Lightweight and modular
- Built-in development server and debugger
- Jinja2 templating engine
- Secure cookie handling for sessions
- RESTful request handling
- Extensions for database integration, authentication, and more

Installing Flask

Flask can be installed using pip:

```
pip install flask
```

Creating a Flask Application

A simple Flask web application:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')

def home():

    return "Hello, Flask!"

if __name__ == '__main__':

    app.run(debug=True)
```

Running the Flask app:

```
python app.py
```

The app will be accessible at <http://127.0.0.1:5000/>.

Flask Project Structure

A typical Flask project consists of:

```
my_flask_app/
```

```
|— app.py
```

```
|— static/
```

```
|— templates/
```

```
|— config.py
```

```
|— requirements.txt
```

- **app.py**: The main application file
- **static/**: Contains static files (CSS, JavaScript, images)
- **templates/**: Contains HTML templates
- **config.py**: Stores configuration settings
- **requirements.txt**: Lists dependencies

Flask Routing

Routing defines URL patterns for the application:

```
@app.route('/about')
```

```
def about():
```

```
    return "About Page"
```

Dynamic routing with parameters:

```
@app.route('/user/<name>')
```

```
def user_profile(name):
```

```
    return f"User: {name}"
```

Flask Templates

Flask uses Jinja2 templating for dynamic HTML rendering:

```
<!-- templates/index.html -->
```

```
<html>
```

```
<body>
```

```
    <h1>Welcome, {{ name }}</h1>
```

```
</body>
```

```
</html>
```

Rendering the template in Flask:

```
from flask import render_template
```

```
@app.route('/welcome/<name>')
```

```
def welcome(name):
```

```
    return render_template('index.html', name=name)
```

Flask Forms and User Input

Flask supports form handling using Flask-WTF:

```
from flask_wtf import FlaskForm

from wtforms import StringField, PasswordField

from wtforms.validators import InputRequired

class LoginForm(FlaskForm):

    username = StringField('Username', validators=[InputRequired()])

    password = PasswordField('Password', validators=[InputRequired()])
```

Flask Database Integration

Flask supports databases like SQLite and PostgreSQL using SQLAlchemy:

```
from flask_sqlalchemy import SQLAlchemy

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database.db'

db = SQLAlchemy(app)
```

Defining a database model:

```
class User(db.Model):

    id = db.Column(db.Integer, primary_key=True)

    username = db.Column(db.String(80), unique=True, nullable=False)
```

Flask Security and Authentication

Flask provides secure authentication mechanisms using Flask-Login:

```
from flask_login import LoginManager

login_manager = LoginManager()

login_manager.init_app(app)
```

7.3 DATABASE

A database is a systematically organized collection of data that is stored electronically in a structured format, enabling easy access, management, and modification. It serves as the backbone of modern applications by allowing users and software systems to efficiently store large volumes of information and retrieve it as needed. Data in a database can include anything from user profiles, transactions, and financial records to logs, multimedia content, and application-specific information.

To manage and interact with this data effectively, databases are controlled through **Database Management Systems (DBMS)**. A DBMS provides the necessary tools and functionalities to define data structures, insert and update records, enforce rules for data integrity, and secure the information from unauthorized access. It also ensures that multiple users or processes can access and manipulate the data simultaneously without conflicts, maintaining consistency and accuracy.

Databases play a critical role across all domains, from small-scale mobile apps to enterprise-level systems. They ensure reliability, scalability, and accuracy of information handling, making them essential for decision-making, analytics, automation, and day-to-day operations in nearly every industry.

SQLite3:

SQLite3 is a lightweight, serverless, and self-contained relational database management system (RDBMS) designed to provide a simple yet powerful solution for managing structured data. Unlike traditional database systems such as MySQL or PostgreSQL, which require a separate server process to run and communicate with applications, SQLite operates without a dedicated server. Instead, the entire database is stored in a single cross-platform file, which makes it highly portable, easy to deploy, and convenient to share or back up.

SQLite3 fully supports **Structured Query Language (SQL)**, enabling developers to perform standard operations such as creating tables, inserting, updating, deleting, and retrieving data. It also incorporates advanced features like **transactions**, ensuring that operations are executed reliably with full support for atomicity, consistency, isolation, and durability (ACID properties). Additionally, it offers indexing for faster query performance and enforces **data integrity** through constraints and rules.

Due to its minimal setup requirements and zero-configuration nature, SQLite3 has become one of the most widely adopted databases for applications that require efficient local storage. It is commonly used in **embedded systems, IoT devices, desktop software, and mobile platforms** such as Android and iOS. Many popular web browsers, operating systems, and enterprise tools also rely on SQLite internally to manage data.

Its simplicity, combined with reliability and speed, makes SQLite3 especially suitable for **small to medium-scale applications** where ease of use and reduced overhead are more important than handling extremely large datasets or high levels of concurrent access. Furthermore, being open-source and free to use in both personal and commercial projects, SQLite3 is considered a go-to solution for developers seeking a stable, secure, and portable database system.

Security

Security is a key consideration for the database. SQLite3 integrates well with application-level encryption techniques, ensuring that sensitive data such as user passwords and admin credentials are protected. Best practices include:

- **Password Hashing & Encryption** before storage.
- **Access Control** to prevent unauthorized data retrieval.
- **Data Integrity Constraints** to avoid corruption or duplication.

These practices protect the system from threats like data leaks, unauthorized access, and tampering.

Efficiency

SQLite3 is known for being **lightweight, fast, and serverless**, which significantly reduces system complexity. Its benefits include:

- **No Dedicated Server Requirement** – The database operates within the application, simplifying deployment.
- **High Read/Write Performance** – Suitable for real-time operations like user login and quick predictions.
- **Low Overhead** – Runs efficiently even on devices with limited resources, making the system more cost-effective.

This efficiency ensures smooth functioning, even when handling frequent user interactions.

Scalability

Although SQLite3 is lightweight, it supports considerable scalability for medium-sized applications. It can:

- Handle **large datasets** required for predictive analytics and model training.
- Scale to support multiple concurrent users through proper connection management.

- Store **historical prediction results**, ensuring that the database can grow with the application's usage over time.

If the system expands significantly in the future, SQLite3 can serve as a stepping stone for migration to more robust databases (e.g., PostgreSQL or MySQL).

Advantages of SQLite3

1. **Serverless and Zero Configuration** – No need for installation or separate database server; the database runs directly from a single file.
2. **Lightweight** – Minimal storage space and memory usage, making it ideal for mobile and embedded applications.
3. **Portable** – Entire database is stored in a single cross-platform file, easy to share, back up, or move.
4. **Fast and Reliable** – Provides efficient read/write operations with full support for transactions and data integrity.
5. **Standard SQL Support** – Implements most of the SQL-92 standard, making it easy to use for developers familiar with SQL.
6. **Scalable for Small to Medium Applications** – Handles moderate amounts of data efficiently without requiring complex infrastructure.
7. **Public Domain (Free to Use)** – Open-source and free for commercial or personal use.

7.4 DEEP LEARNING & MACHINE LEARNING

Deep learning is a part of machine learning that has practical experience in demonstrating and taking care of muddled issues with artificial neural networks. It draws inspiration from the composition and operations of the human brain, particularly from the networked layers of neurons. Because deep learning can automatically learn hierarchical representations from data, it has become very popular and successful in many different fields.

Here are some essential details regarding deep learning:

1. **Neural Networks:** Neural networks, which are made up of layers of connected nodes or artificial neurons, are the foundation of deep learning. The term "deep" in deep learning describes these networks' depth, which indicates that they have several layers (deep architectures).
2. **Deep Neural Networks (DNNs):** Training deep neural networks, which can have several layers (deep networks) or just a few (shallow networks), is a common step in deep learning. The network can extract

complex features and representations from the input data thanks to the depth.

3. Representation Learning: Automatic feature extraction and representation learning are two areas in which deep learning shines. During training, the model learns to extract pertinent features from the data rather than manually engineering features.

4. Backpropagation Training: Backpropagation is a technique used in the training of deep neural networks. The process entails making iterative adjustments to the weights of connections among neurons in order to reduce the discrepancy between the target and the predicted output. Usually, stochastic gradient descent and other optimization algorithms are used for this.

5. Architectures: Different deep learning structures are open, including Transformers for regular language handling, Recurrent Neural Networks (RNNs) for consecutive information, and Convolutional Neural Networks (CNNs) for picture related tasks. These designs are made to deal with specific tasks and data types.

Two well-liked deep learning architectures, ResNet (Residual Network) and DenseNet (Densely Connected Convolutional Network), were created expressly to solve training difficulties for extremely deep neural networks. Let's examine each in brief:

ResNet (Residual Network):

1. Introduction: In 2015, Kaiming He et al. presented ResNet. The use of residual blocks, which have shortcut connections that bypass one or more layers, is the main innovation. This facilitates the training of extremely deep networks and aids in solving the vanishing gradient issue.

2. Shortcut Connections: The introduction of shortcut connections, also known as skip connections, that elide one or more layers, is the fundamental concept of ResNet. Deeper networks can be trained more easily thanks to these connections, which make the gradient flow more freely during backpropagation.

3. Architecture: A collection of residual blocks usually makes up a ResNet architecture. Multiple convolutional layers are present in each block, and the block's output is enhanced by the shortcut connections by the original input.

4. Benefits: ResNet has proven effective in training incredibly complex networks with hundreds of layers. It has been extensively employed in computer vision tasks such as object detection and image

classification.

DenseNet (Densely Connected Convolutional Network):

1. Introduction: DenseNet adopts a different strategy by encouraging dense connectivity between layers. It was first presented by Gao Huang et al. in 2017. Every layer in a DenseNet gets input from every layer that came before it, and every layer that comes after it gets its output.

2. Dense Blocks: The dense block is the cornerstone of DenseNet architecture. It is composed of several layers, where input is provided by the feature maps from all previous layers. Gradient flow is encouraged and feature reuse is improved by this dense connectivity.

3. Transition Blocks: Transition blocks are used to reduce the number of feature maps before passing them to the next dense block, thereby managing the growth of parameters and computation in dense blocks. This preserves the effectiveness of computation.

4. Benefits: DenseNet often requires fewer parameters compared to traditional architectures, leading to more efficient models. The dense connectivity also helps in mitigating the vanishing gradient problem and encourages feature reuse.

5. Applications: DenseNet has demonstrated competitive performance with fewer parameters when used successfully for tasks like image segmentation and classification.

Training Dataset

The training dataset, used to optimize or train the machine learning model, is the largest subset (in terms of size) of the first dataset. In both cases, ML algorithms are tasked with preparing the information, showing them how to generate expectations for the work that needs to be done.

Test Dataset

Now is the right time to test the model utilizing the test dataset after we've prepared it utilizing the preparation dataset. This dataset evaluates the model's presentation and confirms that it can actually sum up to new or untested datasets. Separate from the preparation dataset, the test dataset is an extra subset of the first information. It does, notwithstanding, use it as a benchmark for model assessment after model preparation is done due to its to some degree comparable element types and class likelihood

dispersion. Test data is a structured dataset that includes information for every possible scenario related to a particular issue that the model might encounter in practice. For machine learning projects, the test dataset typically comprises 20–25% of the total original data.

CONVOLUTIONAL NEURAL NETWORK :

Convolutional Neural Networks (CNNs) are a type of deep learning algorithm commonly used for image and video recognition. Here's a brief overview of the working process of a CNN:

1. Input Layer:

Takes in the raw input data, which is usually an image in the case of computer vision tasks.

2. Convolutional Layers:

- Apply a set of filters (kernels) to the input data, performing convolution operations.
- Filters detect patterns and features in the input, such as edges, textures, or shapes.
- Convolution helps preserve spatial relationships within the data.

3. Activation Function:

- Introduces non-linearity to the system, typically using functions like ReLU (Rectified Linear Unit).
- Helps the network learn complex patterns and relationships.

4. Pooling (Subsampling) Layers:

- Reduce the spatial dimensions of the input volume.
- Common pooling methods include max pooling or average pooling.
- Helps decrease computational complexity and reduces the risk of overfitting.

5. Flattening:

- Transform the multi-dimensional data into a one-dimensional vector.
- Prepares the data for input into fully connected layers.

6. Fully Connected Layers:

- Neurons in a layer are connected to every neuron in the adjacent layer.
- Learn complex relationships in the data.
- Typically used in the final layers of the network.

7. Output Layer:

- Produces the final predictions based on the learned features and patterns.
- The activation function in the output layer depends on the task (e.g., softmax for classification).

8. Loss Function:

- Measures the difference between the predicted output and the actual target.
- The goal is to minimize this difference during training.

9. Backpropagation:

- Calculates the gradient of the loss function with respect to the weights.
- Adjusts the weights using optimization algorithms (e.g., stochastic gradient descent) to minimize the loss.

10. Training:

- Iteratively updates the network's parameters using backpropagation and optimization.
- Continues until the model performs well on the training data.

11. Testing/Prediction:

- The trained model is used to make predictions on new, unseen data.

MODEL SELECTION IN Deep Learning:

The process of choosing the optimal algorithm and model architecture for a given task or data set is called model selection in deep learning. It involves comparing and evaluating multiple models to determine which model best fits the data and provides the best results. When choosing a model, considerations such as model complexity, data processing capabilities, and ability to generalize to new examples are taken into account. Indicators like accuracy and mean squared error are used along with techniques like grid search and cross-validation to assess and compare models. The goal of model selection is to identify a model that strikes a balance between performance and complexity in order to generate solid predictions and strong generalization capabilities.

MACHINE LEARNING

Machine Learning is one of the booming technologies across the world that enables computers/machines to turn a huge amount of data into predictions. However, these predictions highly depend on the quality of the data, and if we are not using the right data for our model, then it will not generate the expected result. In machine learning projects, we generally divide the original dataset into training data and test data. We train our model over a subset of the original dataset, i.e., the training dataset, and then evaluate whether it can generalize well to the new or unseen dataset or test set. Therefore, train and test datasets are the two key concepts of machine learning, where the training dataset is used to fit the model, and the test dataset is used to evaluate the model.

Training Dataset

The training data is the biggest (in -size) subset of the original dataset, which is used to train or fit the machine learning model. Firstly, the training data is fed to the ML algorithms, which lets them learn how to make predictions for the given task.

Test Dataset

Once we train the model with the training dataset, it's time to test the model with the test dataset. This dataset evaluates the performance of the model and ensures that the model can generalize well with the new or unseen dataset. The test dataset is another subset of original data, which is independent of the training dataset. However, it has some similar types of features and class probability distribution and uses it as a benchmark for model evaluation once the model training is completed. Test data is a well-organized dataset that contains data for each type of scenario for a given problem that the model would be facing when used in the real world. Usually, the test dataset is approximately 20-25% of the total original data for an ML project.

RANDOM FOREST ALGORITHM:

Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of ensemble learning, which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model.

As the name suggests, "Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset." Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.

The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

MODEL SELECTION IN MACHINE LEARNING:

Model selection in machine learning is the process of selecting the best algorithm and model architecture for a specific job or dataset. It entails assessing and contrasting various models to identify the one that best fits the data & produces the best results. Model complexity, data handling capabilities, and generalizability to new examples are all taken into account while choosing a model. Models are evaluated and contrasted using methods like cross-validation, and grid search, as well as indicators like accuracy and mean squared error. Finding a model that balances complexity and performance to produce reliable predictions and strong generalization abilities is the aim of model selection.

7. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner.

7.1 Types of Testing

Unit testing

Unit testing focuses on testing individual components in isolation. In the weather AI implementation, unit tests would examine each method separately - the model building function, data generation method, and prediction function. The behavior would involve creating mock inputs and verifying that each component produces expected outputs with correct data types and shapes. For example, testing the `build_model()` method would verify that it creates a model with exactly 2 inputs, 3 outputs, and proper layer configurations. Unit tests would catch issues like incorrect tensor dimensions, wrong activation functions, or missing layers early in development. However, the current implementation lacks proper unit tests, making it difficult to identify which specific component fails when errors occur.

Integration testing

Integration testing examines how different components work together. In this weather AI system, integration tests would verify that the CNN (processing satellite images) and LSTM (processing meteorological time series) components properly combine their features before feeding into the final prediction layers. The behavior would involve testing data flow between layers, ensuring that the concatenation of CNN and LSTM features works correctly, and verifying that all three output heads receive the combined features appropriately. The current implementation performs basic integration testing by running the complete prediction pipeline, but it doesn't validate intermediate data transformations or check for proper gradient flow during training.

System Testing

System testing evaluates the complete integrated system in an environment that closely mimics production conditions. For the weather AI, this would involve testing the entire workflow from data ingestion through model training to final prediction output, including the visualization components. The behavior would encompass testing the training process with large datasets, verifying that the

model saves and loads correctly, ensuring that the plotting functions work with various prediction outcomes, and validating that the system performs adequately under realistic computational constraints. The current implementation includes basic system testing through the `main()` function, but lacks comprehensive testing of different deployment scenarios or performance under various load conditions.

White Box Testing

White box testing involves examining the internal structure and logic of the code with full knowledge of the implementation. For the weather AI, this would mean testing specific algorithmic choices like the spiral storm pattern generation, the pressure-based storm classification logic, and the mathematical relationships used in synthetic data creation. The behavior would involve verifying that the storm intensity classification correctly maps pressure ranges to storm categories, testing edge cases in the spiral generation algorithm, and ensuring that the synthetic meteorological data follows realistic patterns. White box testing would reveal issues like incorrect mathematical formulas, improper boundary conditions, or logical errors in the classification rules that might not be apparent from external testing alone.

Black Box Testing

Black box testing treats the system as a "black box" without knowledge of internal implementation, focusing only on inputs and outputs. For the weather prediction AI, black box testing would involve providing various satellite images and meteorological data combinations and evaluating whether the predictions are reasonable without caring how the model arrives at those conclusions. The behavior would include testing with different storm intensities, various weather conditions, and edge cases like extremely high or low pressure systems. Testers would verify that stronger storms produce higher risk levels, that predictions fall within realistic ranges, and that the system handles invalid inputs gracefully. This testing method would catch issues where the model produces nonsensical predictions despite having correct internal logic.

Regression Testing

Regression testing ensures that new changes don't break existing functionality. For the weather AI, this would involve maintaining a suite of test cases with known input-output pairs and running them after each model modification or code update. The behavior would include comparing new prediction results against previously validated outputs, testing that model architecture changes don't alter fundamental prediction capabilities and ensuring performance optimizations don't introduce accuracy

regressions. The current implementation doesn't include regression testing, making it risky to modify the model or update the codebase without potentially introducing new bugs or degrading prediction quality.

Performance Testing

Performance testing evaluates how the system behaves under various load conditions and measures response times, memory usage, and computational efficiency. For the weather AI, this would involve testing prediction latency with different input sizes, memory consumption during training with large datasets, and model accuracy degradation with increased processing speed requirements. The behavior would include measuring how quickly the system can process satellite images of varying resolutions, testing memory usage when processing multiple predictions simultaneously, and evaluating training time with different batch sizes. The current implementation doesn't include performance testing, making it difficult to determine if the system would meet real-time prediction requirements in operational weather forecasting scenarios.

Functional Testing

Functional testing verifies that the system meets specified functional requirements. For the weather prediction AI, this would involve testing that the system correctly identifies different storm categories, provides accurate risk assessments, and generates reliable weather parameter predictions. The behavior would include validating that tropical depressions are classified differently from hurricanes, that risk levels correlate appropriately with storm intensity, and that predicted weather parameters (temperature, pressure, humidity, wind speed) fall within meteorologically reasonable ranges. The current implementation lacks explicit functional testing, relying instead on visual inspection of sample predictions.

7.2 : Testcases

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC_01	Upload valid thermal image of Iron	Thermal image (Iron)	System predicts "Iron"	Iron	Pass
TC_02	Upload valid thermal image of Aluminium	Thermal image (Aluminium)	System predicts "Aluminium"	Aluminium	Pass
TC_03	Upload valid thermal image of Copper	Thermal image (Copper)	System predicts "Copper"	Copper	Pass
TC_04	Upload valid thermal image of Brass	Thermal image (Brass)	System predicts "Brass"	Brass	Pass
TC_05	Upload valid thermal image of Zinc	Thermal image (Zinc)	System predicts "Zinc"	Zinc	Pass
TC_06	Upload valid thermal image of Lead	Thermal image (Lead)	System predicts "Lead"	Lead	Pass
TC_07	Upload noisy/blurred thermal image	Low-quality image	System still predicts correct metal or nearest match	Correct/Approximate	Pass
TC_08	Upload image with similar thermal pattern metals (e.g., Brass vs Zinc)	Thermal image	Correct classification using ensemble model	Correct	Pass
TC_09	Upload invalid file format	.txt / unsupported file	Error message displayed	Error shown	Pass
TC_10	Upload multiple images sequentially	Multiple thermal images	System processes each and gives correct predictions	System keeps loading	Fail

TEST CASE 1:

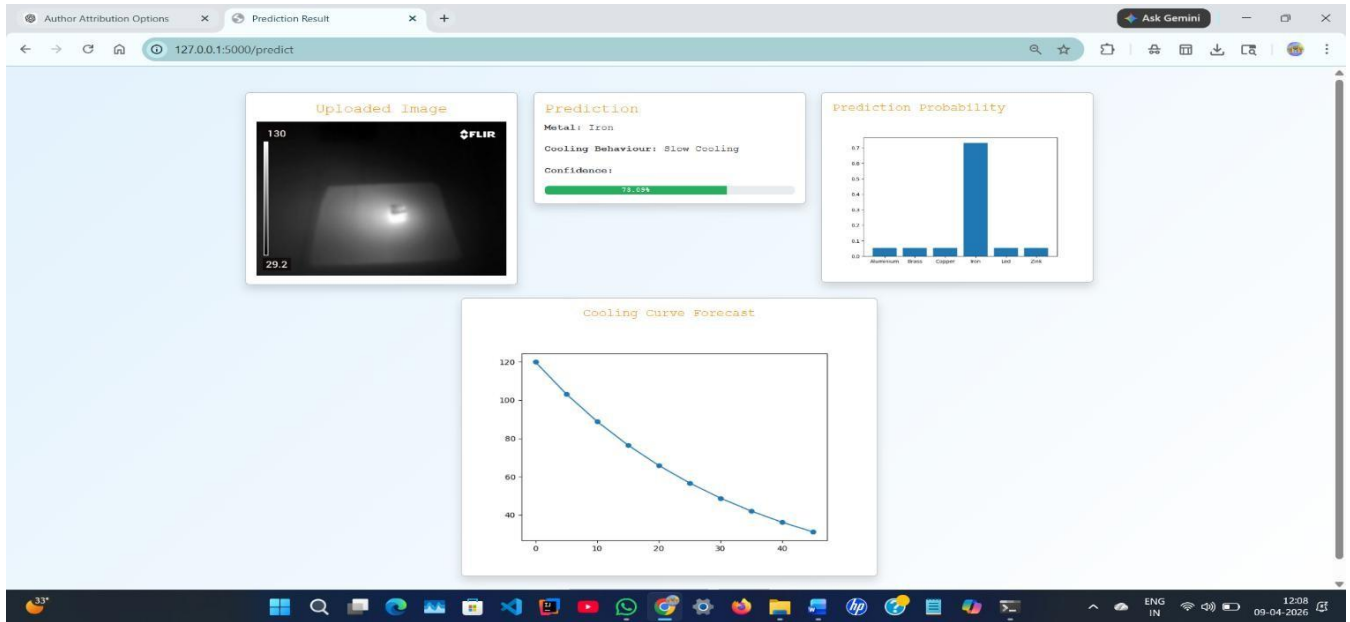


Fig no:7.2.1: Upload valid thermal image of Iron

TEST CASE 2:

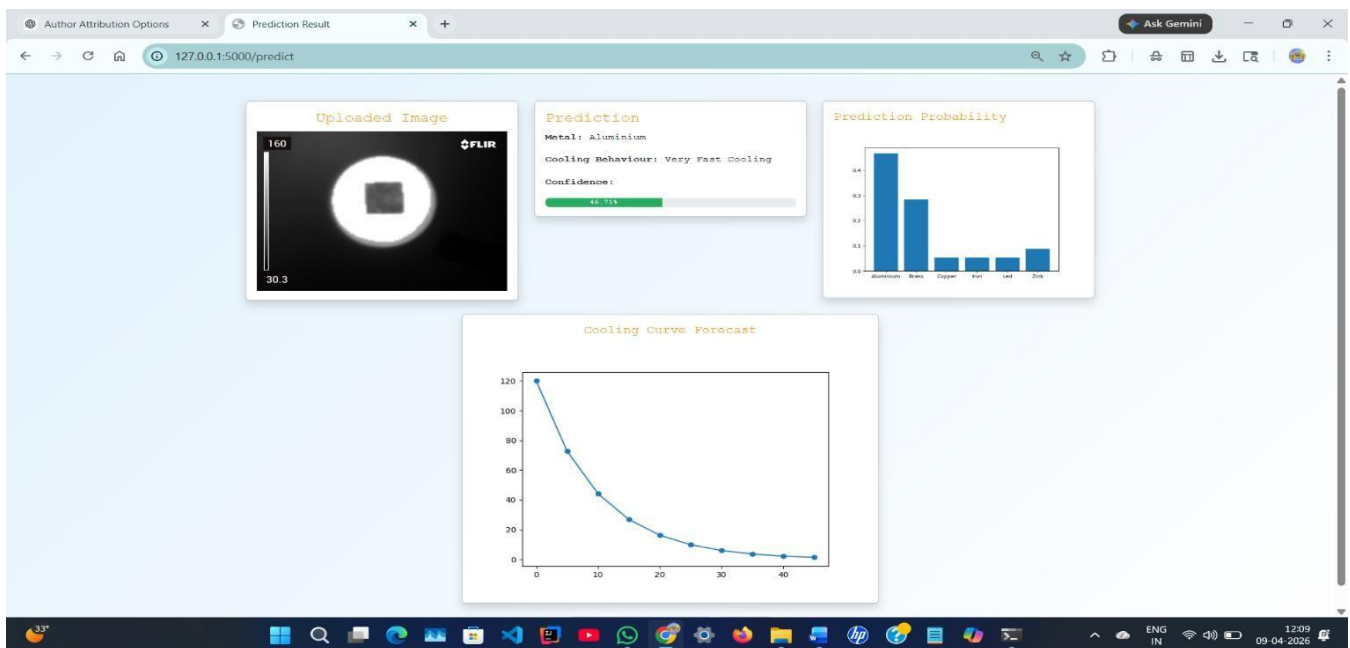


Fig no:7.2.2: Upload valid thermal image of Aluminium

TEST CASE 3:

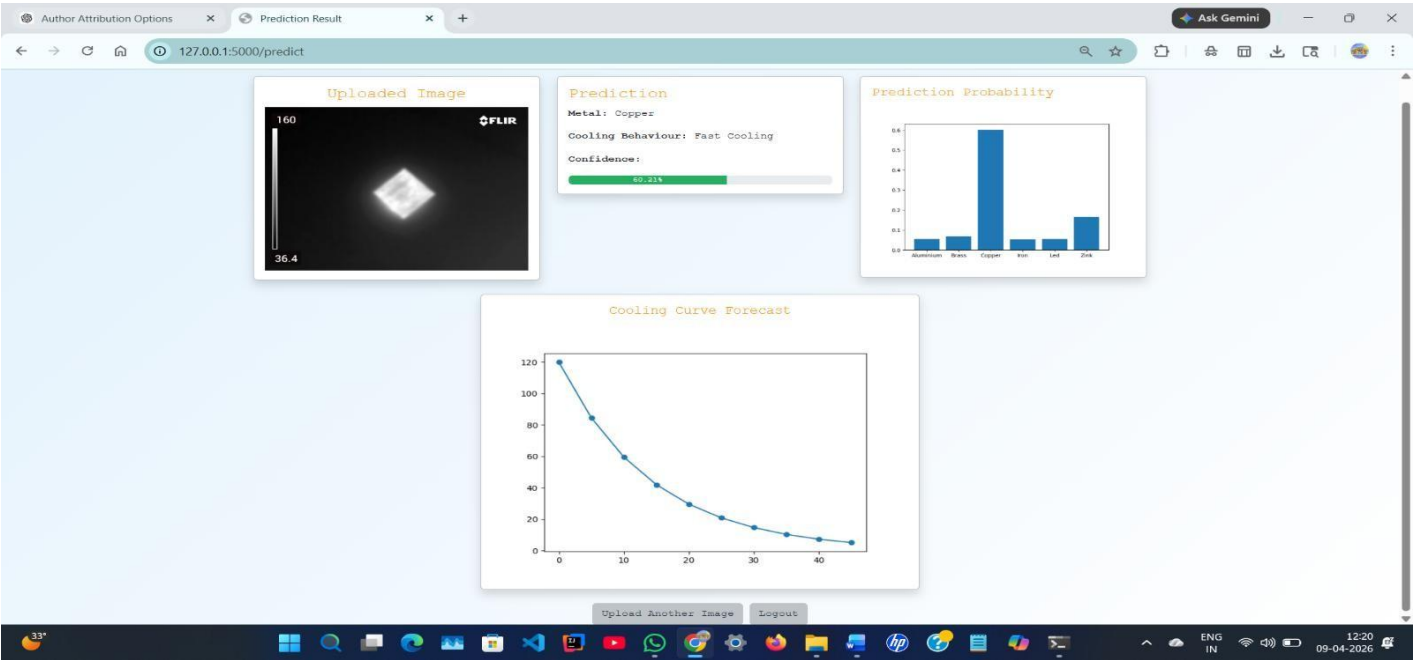


Fig no:7.2.3: Upload valid thermal image of Copper

TEST CASE 4:

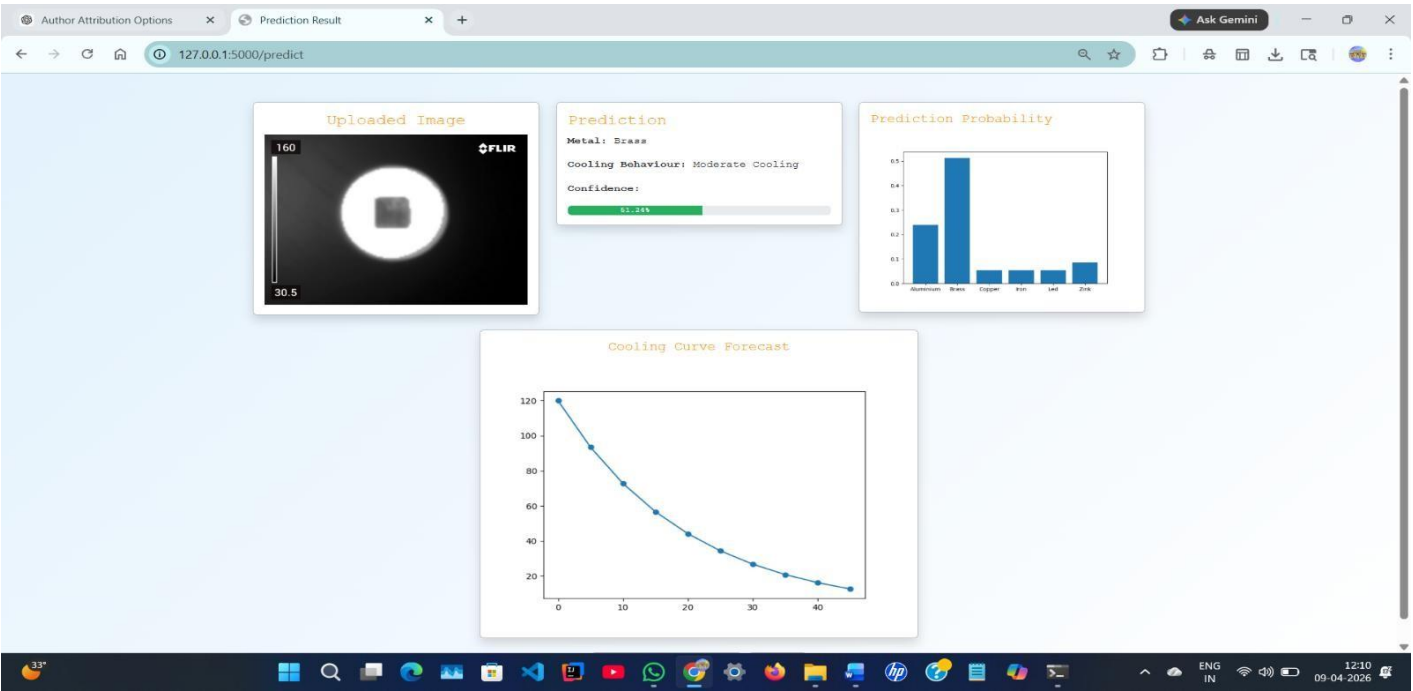


Fig no:7.2.4: Upload valid thermal image of Brass

TEST CASE 5:

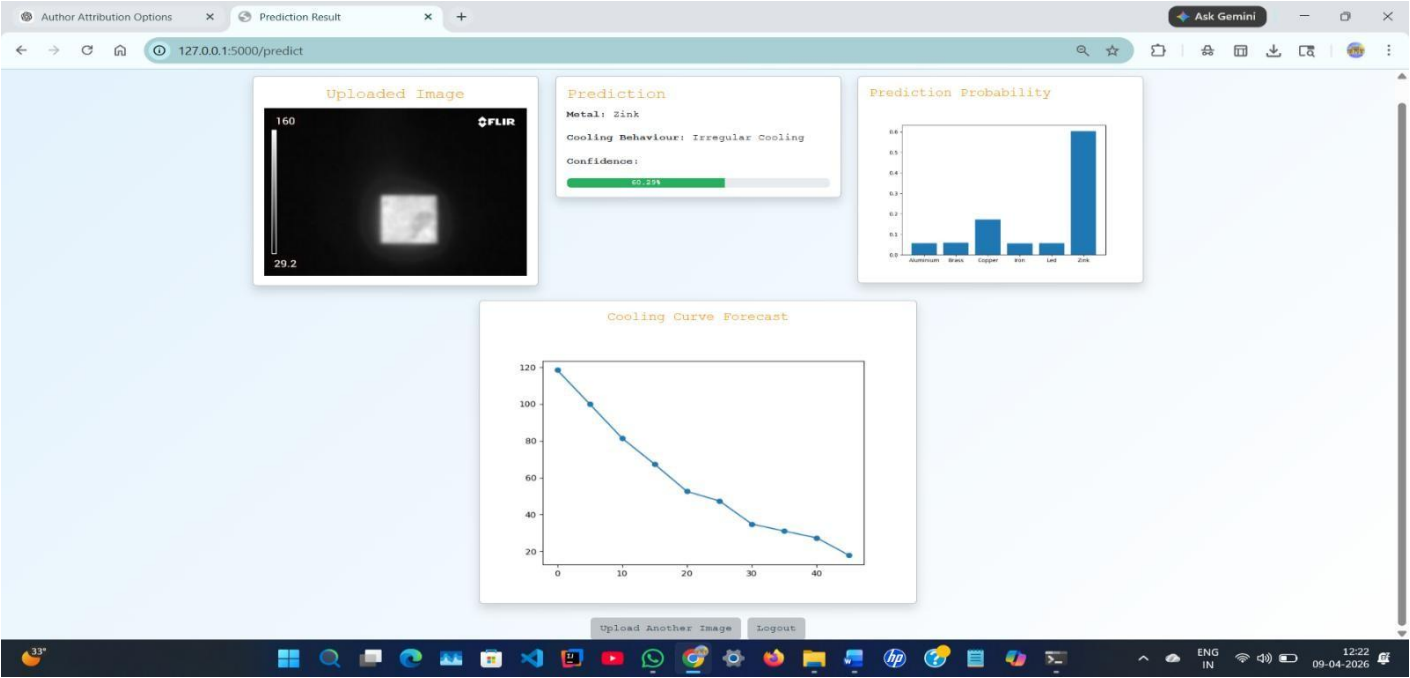


Fig no:7.2.5: Upload valid thermal image of Zinc

TEST CASE 6:

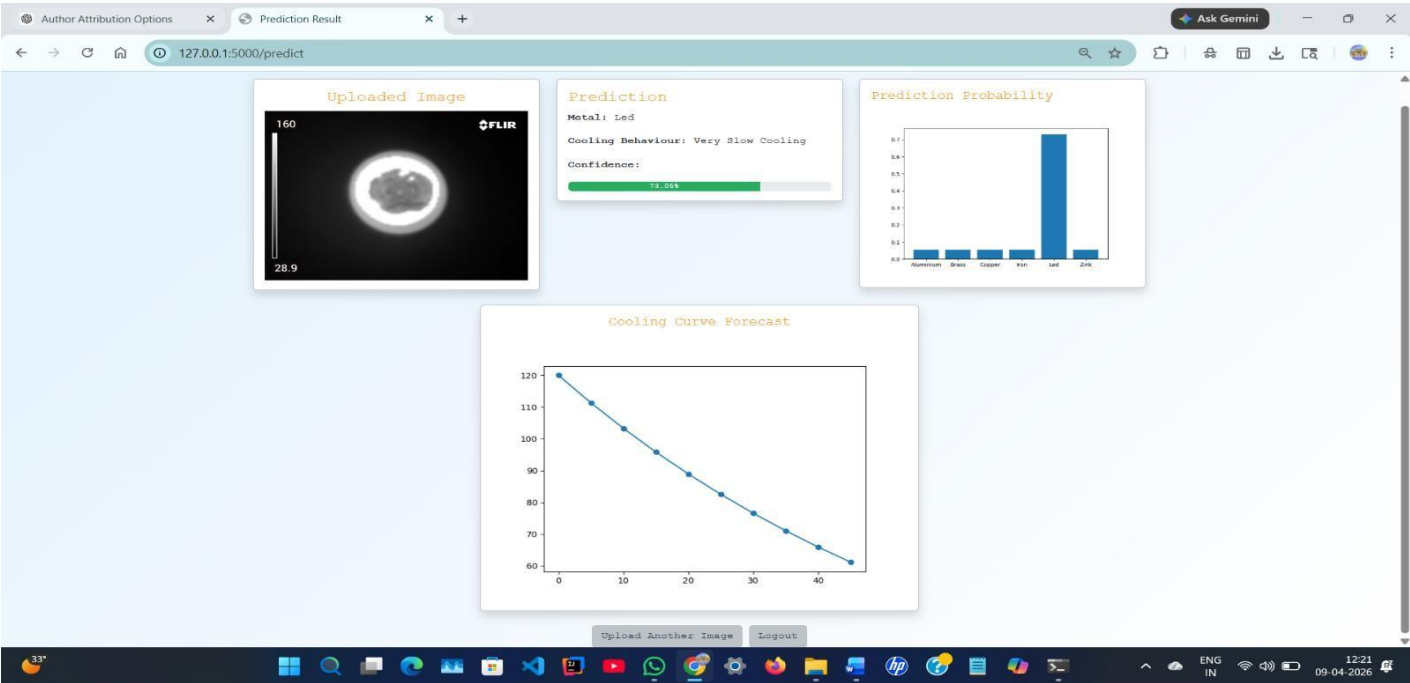


Fig no:7.2.6: Upload valid thermal image of Lead

TEST CASE 7:

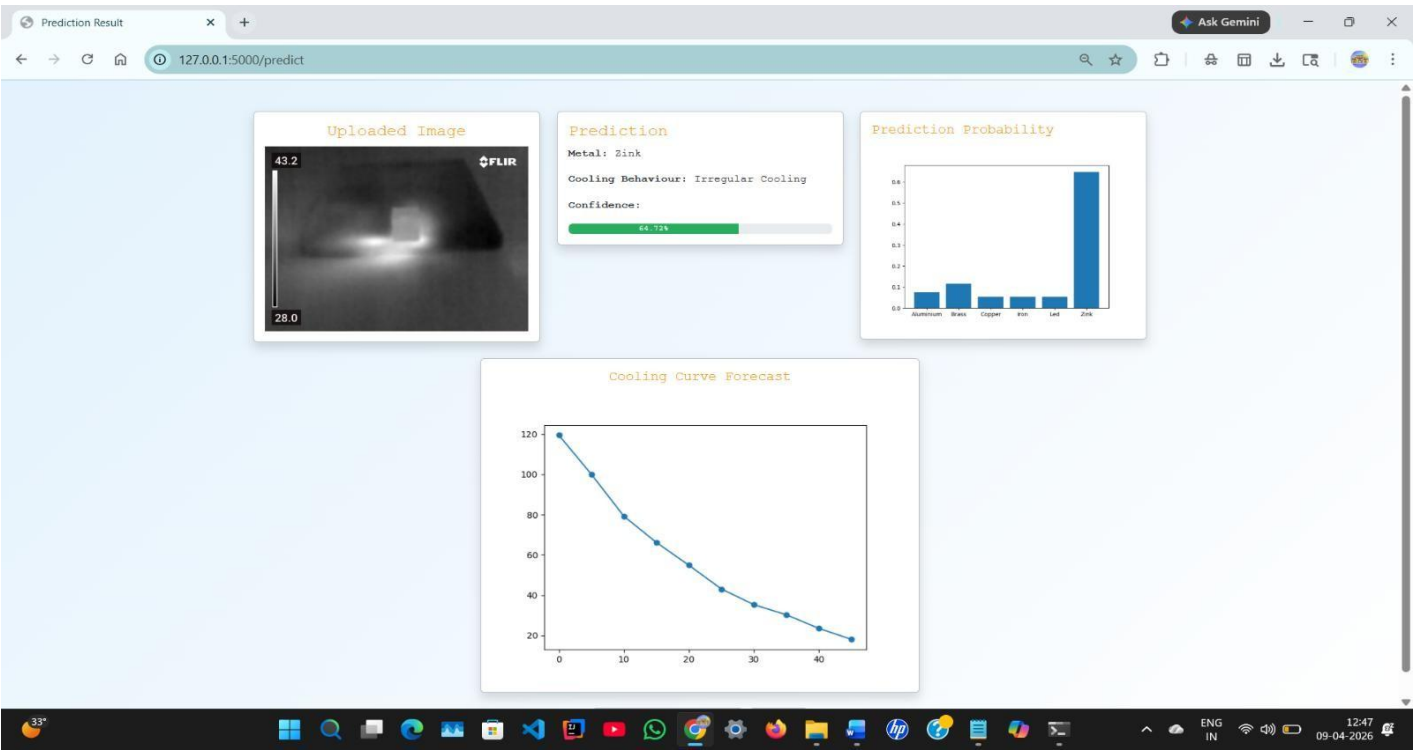


Fig no:7.2.7: Upload noisy/blurred thermal image

TEST CASE 8:

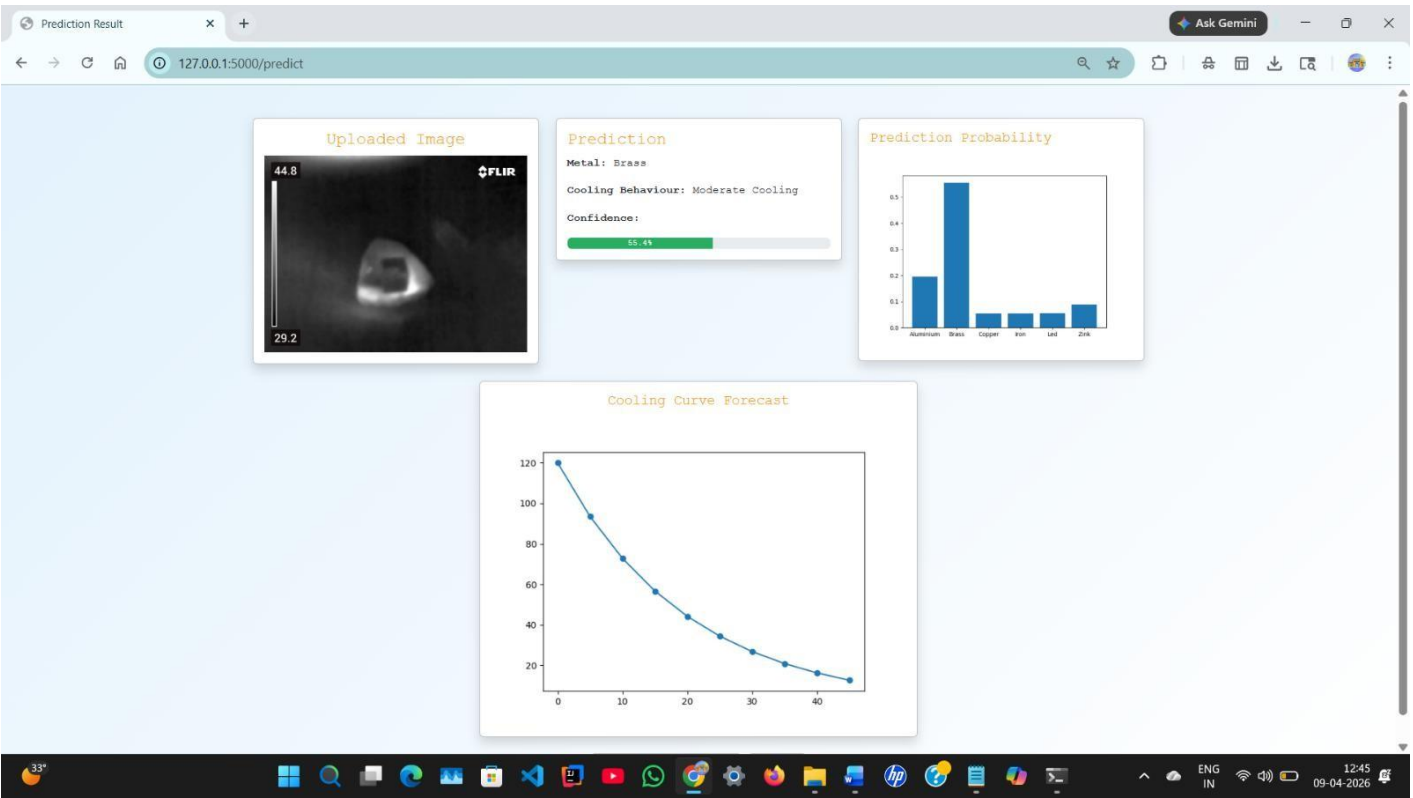


Fig no:7.2.8: Upload image with similar thermal pattern metals (e.g., Brass vs Zinc)

TEST CASE 9:

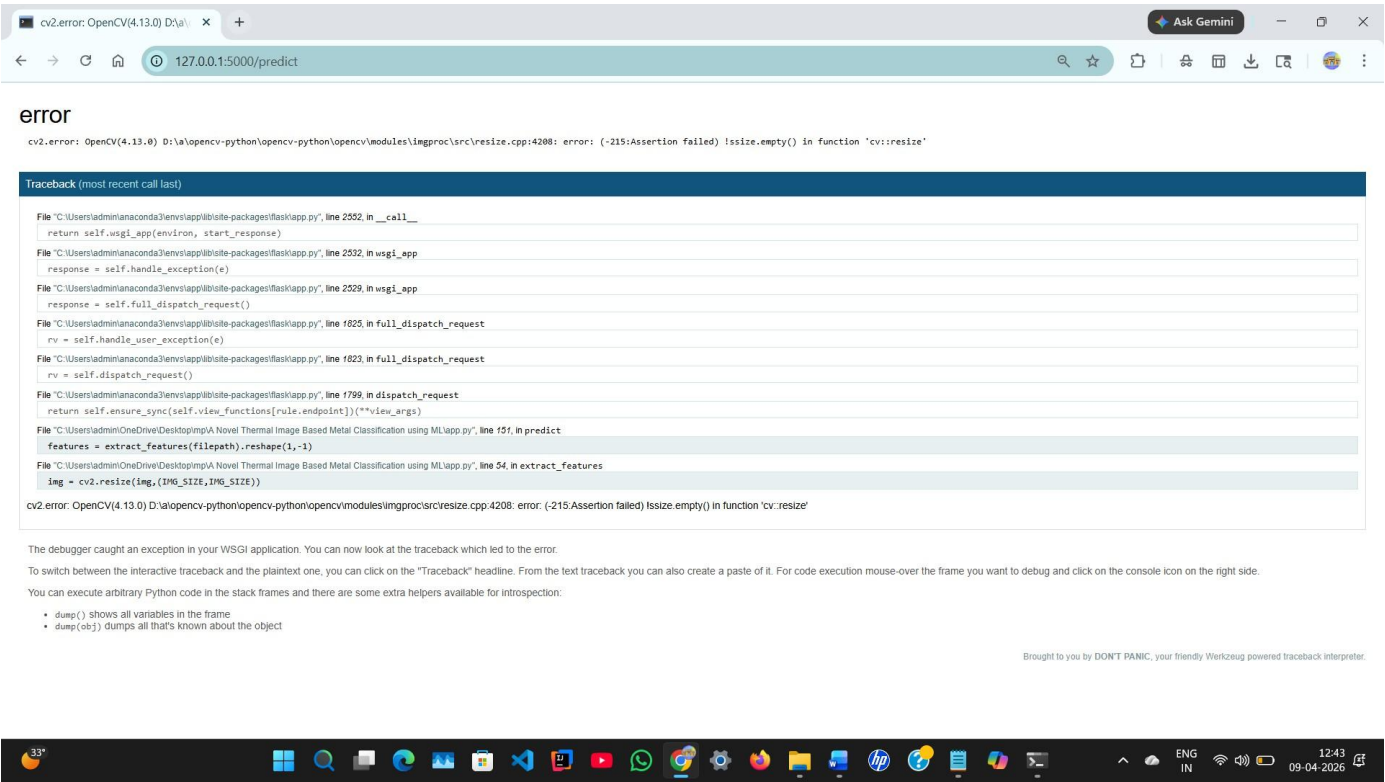


Fig no:7.2.9: Upload invalid file format

TEST CASE 10:

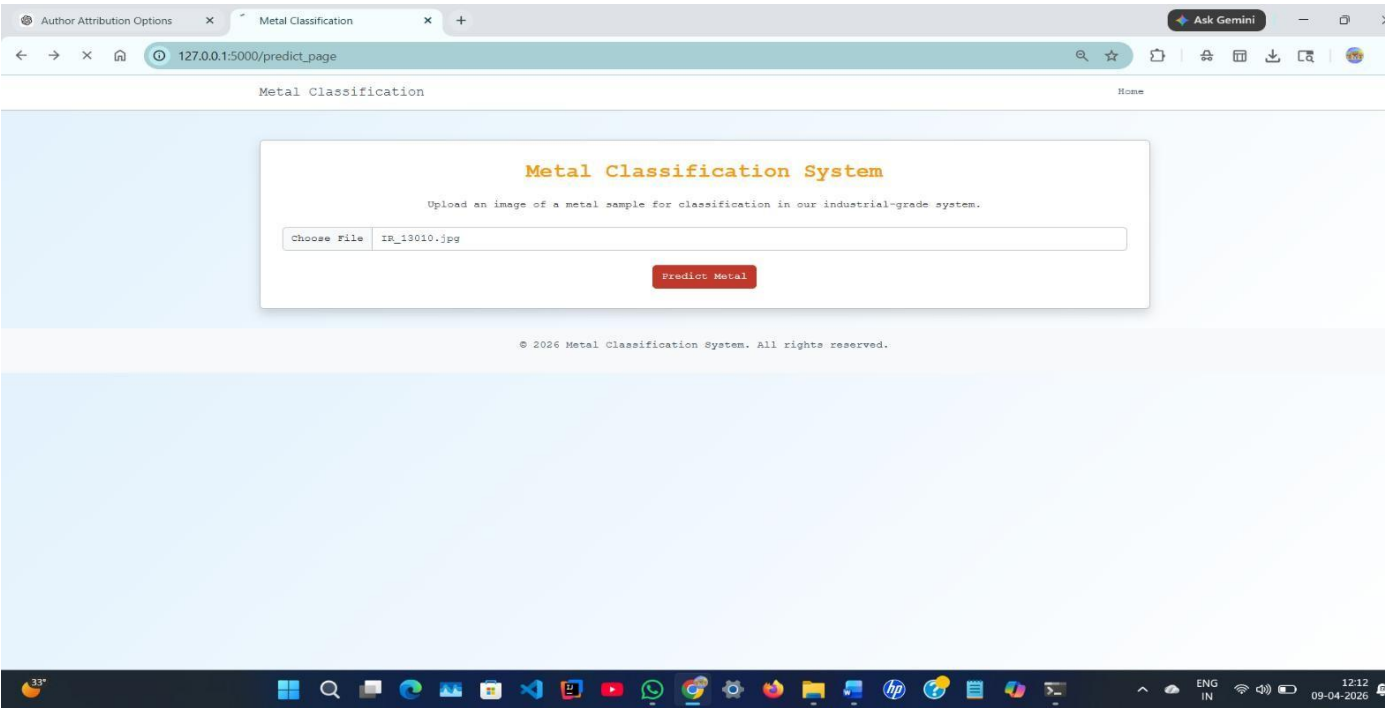


Fig no:7.2.10: Upload multiple images sequentially

8. OUTPUT SCREENS

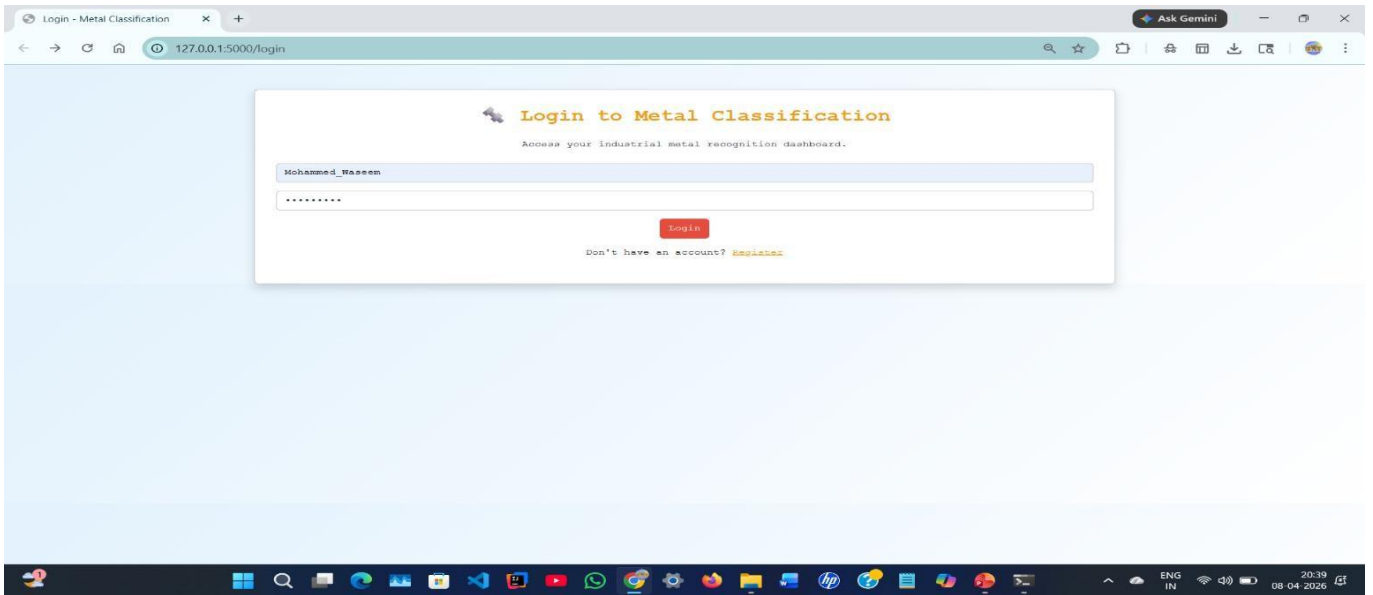


Figure 8.1: Output screen-1

User is asked to login

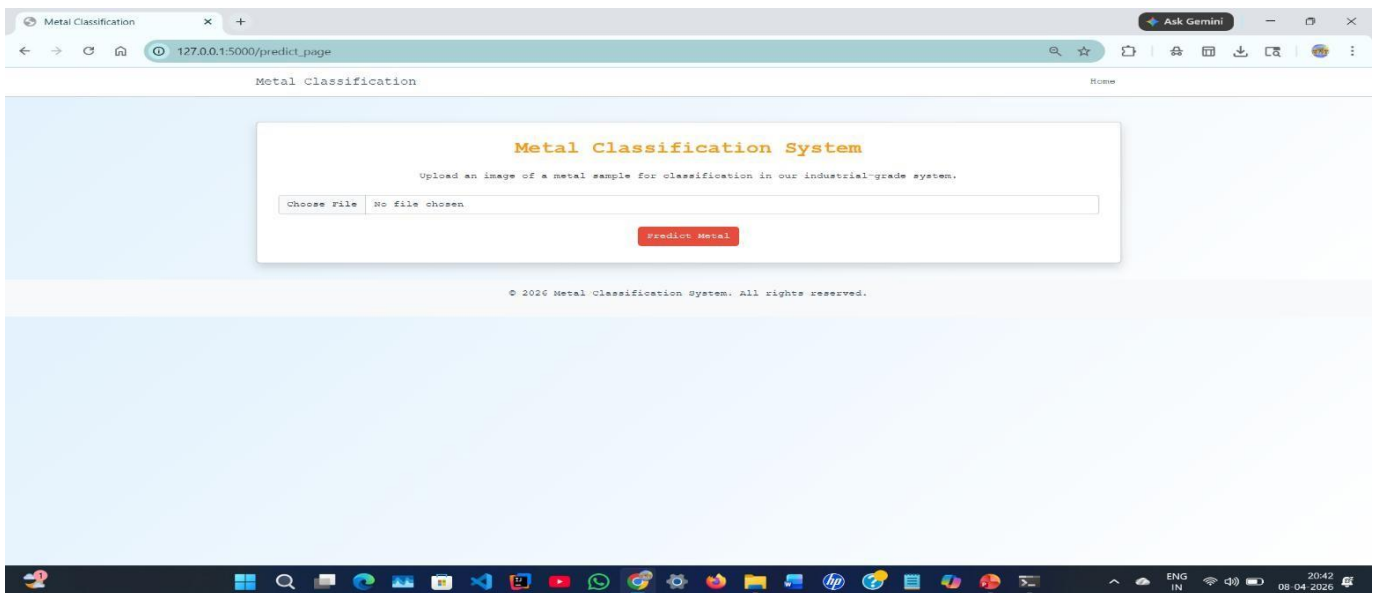


Figure 8.2: Output screen-2

User uploads the thermal image of the metal

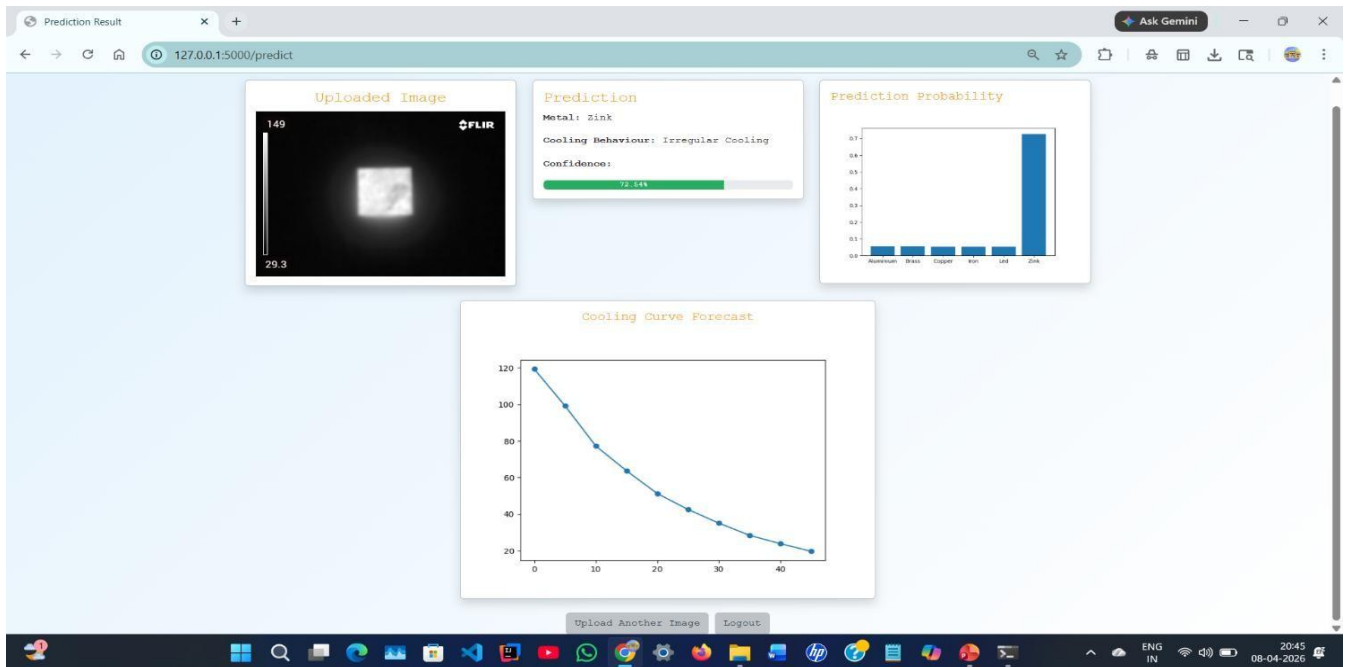


Figure 8.3: Output screen-3

System gives the output for heating image

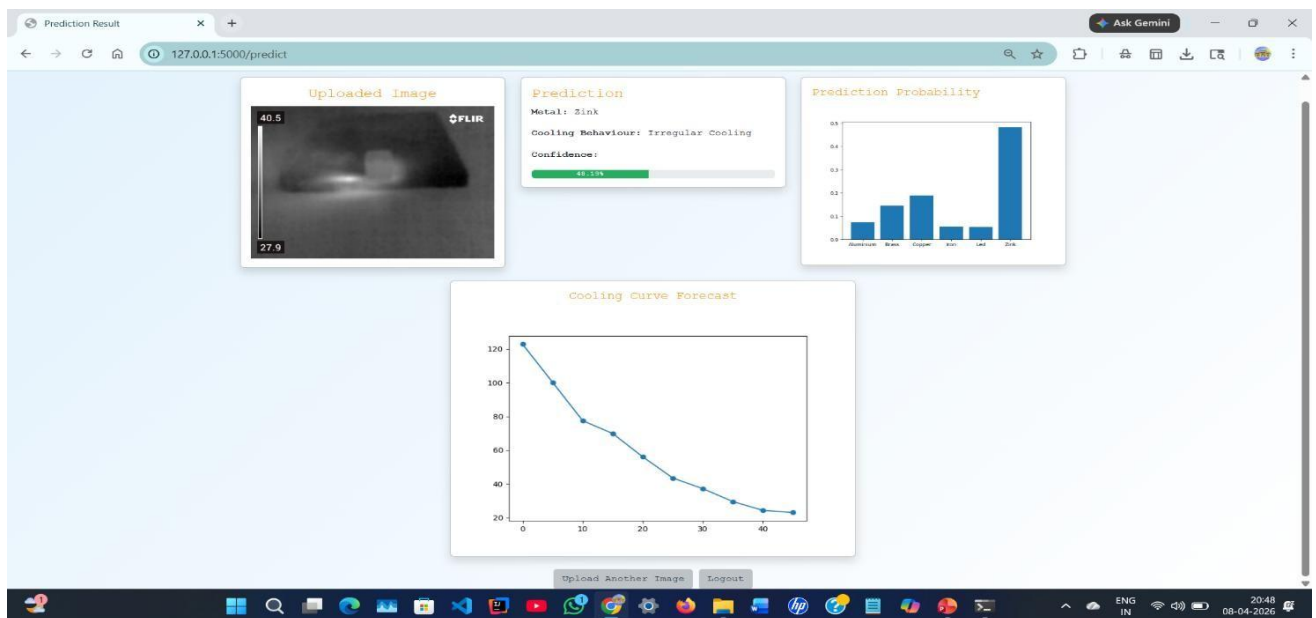


Figure 8.4: Output

System shows different properties for cooling image

9. CONCLUSION

This research presents an advanced metal classification system that integrates thermal imaging, deep learning, and ensemble machine learning techniques to achieve high accuracy and efficiency. By utilizing thermal images captured during heating and cooling processes, the system effectively analyzes the unique thermal behavior of different metals. The use of the EfficientNetB4 model for feature extraction enables the identification of complex spatial patterns, while cooling curve analysis incorporates temporal characteristics, enhancing the system's ability to distinguish between metals with similar properties. Furthermore, the implementation of an ensemble voting classifier combining Logistic Regression, Bagging with Decision Tree, and Boosted Random Forest significantly improves classification performance by reducing overfitting and increasing robustness. The system achieves an accuracy of approximately 99%, outperforming traditional machine learning approaches. The integration of a Flask-based web application ensures real-time usability and practical deployment. Overall, the proposed system provides a cost-effective, scalable, and reliable solution for automated metal classification in industrial applications such as recycling, quality control, and smart manufacturing.

10. FUTURE ENHANCEMENTS

Future Enhancements Although the proposed system demonstrates high accuracy and efficiency, there are several opportunities for further enhancement. Future work can focus on extending the system to handle a wider range of metals, including alloys and composite materials, which are commonly found in real-world industrial environments. Incorporating more diverse datasets with varying shapes, sizes, and surface conditions such as corrosion and coatings can improve the model's generalization capability. Additionally, advanced deep learning architectures and hybrid models can be explored to further enhance feature extraction and classification performance. The integration of real-time IoT-based sensors and edge computing can enable faster and more efficient deployment in industrial settings. Moreover, automating region of interest (ROI) detection and thermal image segmentation can reduce manual intervention and improve consistency. Future research can also focus on incorporating explainable AI techniques to improve model interpretability and user trust. Finally, expanding the system to support mobile or cloud-based applications can make it more accessible and scalable for large-scale industrial and commercial use.

11. REFERENCES

- [1] Y. R. Smith, J. R. Nagel, and R. K. Rajamani, “Eddy current separation for recovery of non-ferrous metallic particles: A comprehensive review,” *Minerals Eng.*, vol. 133, pp. 149–159, Mar. 2019.
- [2] U. Nations. (2015). Sustainable Development Goal 12: Ensure Sustainable Consumption and Production Patterns. Accessed: Jan. 22, 2025. [Online]. Available: <https://sdgs.un.org/goals/goal12>
- [3] N. K. Kladouri, S. Skaltsa, T. Gerodimos, K. Pezouvani, and A. G. Karydas, “Microscopic X-ray fluorescence analyses (μ -XRF) of copper-based and silver alloy coins minted in Rhodes, Greece, from the fourth century BCE to the second century CE,” *Archaeological Anthropological Sci.*, vol. 15, no. 9, p. 141, Sep. 2023.
- [4] T. Yang, Q. Liu, H. Li, Q. Zeng, and L. Chan, “Anthropogenic magnetic particles and heavy metals in the road dust: Magnetic identification and its implications,” *Atmos. Environ.*, vol. 44, no. 9, pp. 1175–1185, Mar. 2010.
- [5] A. Picón, O. Ghita, A. Bereciartua, J. Echazarra, P. F. Whelan, and P. M. Iriondo, “Real-time hyperspectral processing for automatic nonferrous material sorting,” *J. Electron. Imag.*, vol. 21, no. 1, 2012, Art. no. 013018.
- [6] M. D. O’Toole, N. Karimian, and A. J. Peyton, “Classification of nonferrous metals using magnetic induction spectroscopy,” *IEEE Trans. Ind. Informat.*, vol. 14, no. 8, pp. 3477–3485, Aug. 2018.
- [7] K. C. Williams, M. D. O’Toole, L. A. Marsh, and A. J. Peyton, “Classification of batteries in waste streams using magnetic induction spectroscopy,” in *Proc. IEEE Sensors Appl. Symp. (SAS)*, Aug. 2022,
- [8] B. Dekdouk, C. Ktistis, L. A. Marsh, D. W. Armitage, and A. J. Peyton, “Towards metal detection and identification for humanitarian demining using magnetic polarizability tensor spectroscopy,” *Meas. Sci. Technol.*, vol. 26, no. 11, Nov. 2015,
- [9] M. D. O’Toole and A. J. Peyton, “Classification of non-ferrous scrap metal using two component magnetic induction spectroscopy,” in *Proc. IEEE Sensors Appl. Symp. (SAS)*, Mar. 2019,
- [10] K. C. Williams, M. D. O’Toole, and A. J. Peyton, “Scrap metal classification using magnetic induction spectroscopy and machine vision,” *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2023.
- [11] L. Brooks and G. Gaustad, “Positive material identification (PMI) capabilities in the metals secondary industry: An analysis of XRF and LIBS handheld analyzers,” in *Light Metals 2019*. Cham,

Switzerland: Springer, 2019.

- [12] D. Pan, Z. Jiang, Z. Chen, W. Gui, Y. Xie, and C. Yang, “Temperature measurement and compensation method of blast furnace molten iron based on infrared computer vision,” *IEEE Trans. Instrum. Meas.*, vol. 68, no. 10, pp. 3576–3588, Oct. 2019.
- [13] A.-I. García-Moreno, J.-M. Alvarado-Orozco, J. Ibarra-Medina, and E. Martínez-Franco, “Image-based porosity classification in al-alloys by laser metal deposition using random forests,” *Int. J. Adv. Manuf. Technol.*, vol. 110, nos. 9–10, pp. 2827–2845, Oct. 2020.
- [14] S. R. C. S. Rana. (2025). Metal Thermal Pattern. Mendeley Data. [Online]. Available: <https://data.mendeley.com/preview/rhby6j4wr7?a=2ab4b7a5-fc2d-4a14-a848-a9444e51020f>
- [15] J. Deng, J. Tang, F. Guo, Q.-M. Cai, X. Cao, L. Zhou, H. Li, Z.-Y. Du, Y. Li, Y. Zhu, B. Pu, and J. Fan, “A non-destructive metal classification system based on magnetic induction and mutual impedance measurement,” in *Proc. Photon. Electromagn. Res. Symp. (PIERS)*, Apr. 2024,
- [16] J. Lin, P. Dai, C. Che, X. Lin, Y. Li, J. Yang, Y. Huang, Y. Ren, X. Zhen, and X. Yang, “Research on automotive scrap metal classification method using laser-induced breakdown spectroscopy and two-step clustering algorithm,” *J. Laser Appl.*, vol. 36, no. 2, May 2024, Art. no. 022011.
- [17] D. Wang, Z. Zhang, W. Yin, H. Chen, H. Ma, G. Zhou, and Y. Zhang, “Conductivity classification of multi-shape nonmagnetic metal considering spatial position drift effect with a triple-coil electromagnetic sensor,” *Sensors*, vol. 22, no. 15, p. 5694, Jul. 2022.
- [18] A. Daban, S. A. M. Aris, and M. S. M. Anuar, “Ferrous metal classifications based on sparks pattern using CNN pretrained models,” in *Proc. IEEE Symp. Comput. Informat. (ISCI)*, Oct. 2023,
- [19] M. Mazzarisi, A. Angelastro, M. Latte, T. Colucci, F. Palano, and S. L. Campanelli, “Thermal monitoring of laser metal deposition strategies using infrared thermography,” *J. Manuf. Processes*, vol. 85, pp. 594–611, Jan. 2023.
- [20] D. A. Stephenson, “Assessment of steady-state metal cutting temperature models based on simultaneous infrared and thermocouple data,” *J. Eng. for Ind.*, vol. 113, no. 2, pp. 121–128, May 1991.